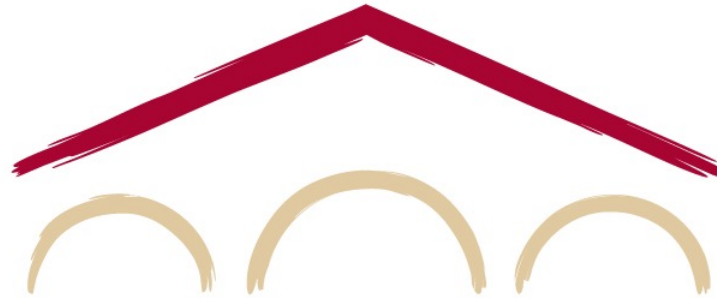


Natural Language Processing with Deep Learning

CS224N/Ling284



Christopher Manning

Lecture 7: Attention and Final Projects; Practical Tips

Lecture Plan

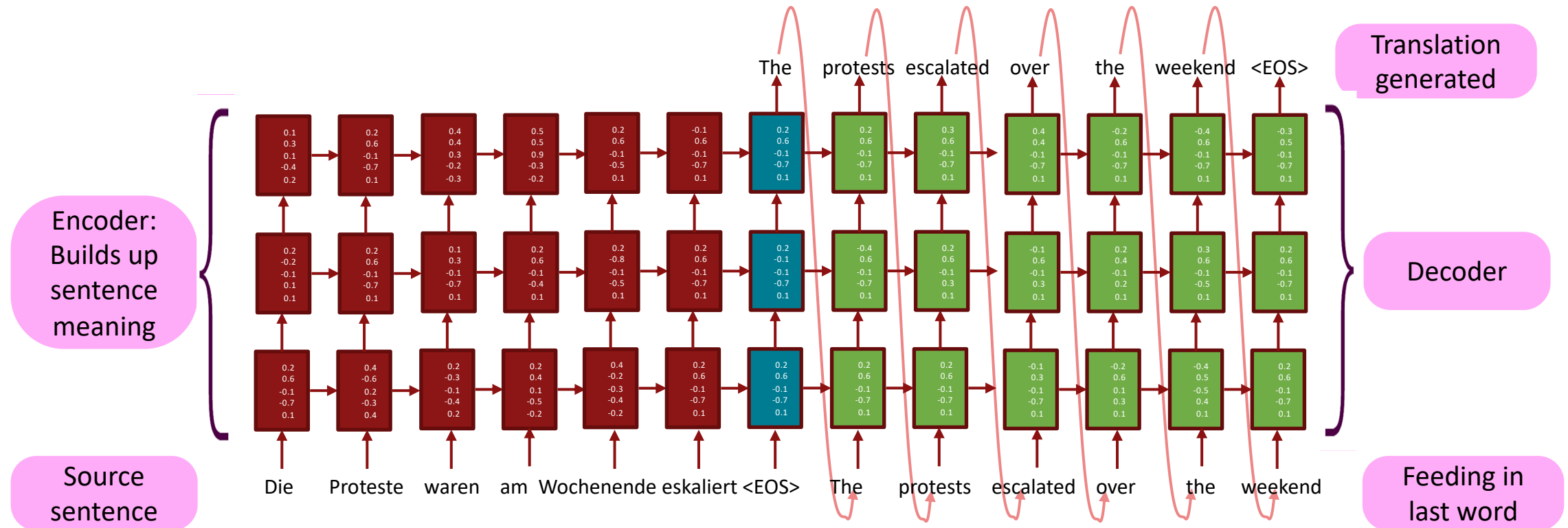
Lecture 7: Attention – final Projects – practical tips!

1. Evaluation of MT [5 mins]
 2. Attention [30 mins]
– Mini Break –
 3. Final projects types and details; assessment revisited [20 mins]
 4. Finding research topics and sources of data [25 mins]
- **Announcements**
 - Final project guide and project proposal details is out today!
 - You get 9 days for it, due Thu; a significant part of it is writing a paper review. Talk to TAs about it.
 - Also make sure that you're underway on Assignment 3, due Tue
 - Get started early! It's bigger – and harder coding-wise – than the previous assignments 😓
 - **Starting with Ass 3, the TAs will no longer look at and debug your code for you!**

1. Multi-layer deep encoder-decoder machine translation net

[Sutskever et al. 2014; Luong et al. 2015]

The hidden states from RNN layer i are the inputs to RNN layer $i + 1$



How do we evaluate Machine Translation?

Commonest way: **BLEU** (**B**ilingual **E**valuation **U**nderstudy)

You'll see BLEU in detail in
Assignment 3!

- BLEU compares the machine-written translation to one or several human-written translation(s), and computes a **similarity score** based on:
 - Geometric mean of **n -gram precision** (usually for 1, 2, 3 and 4-grams)
 - Plus a penalty for too-short system translations
- BLEU is **useful** but **imperfect**
 - There are many valid ways to translate a sentence
 - Therefore, a **good** translation can get a **poor** BLEU score because it has low n -gram overlap with the human translation ☹️

BLEU score against 4 reference translations

But commonly now there is only one references
and so the results are more “in expectation”

Reference translation 1:

The U.S. island of Guam is maintaining a high state of alert after the Guam airport and its offices both received an e-mail from someone calling himself the Saudi Arabian Osama bin Laden and threatening a biological/chemical attack against public places such as the airport.

Reference translation 2:

Guam International Airport and its offices are maintaining a high state of alert after receiving an e-mail that was from a person claiming to be the wealthy Saudi Arabian businessman Bin Laden and that threatened to launch a biological and chemical attack on the airport and other public places .

Machine translation:

The American [?] international airport and its the office all receives one calls self the sand Arab rich business [?] and so on electronic mail , which sends out ; The threat will be able after public place and so on the airport to start the biochemistry attack , [?] highly alerts after the maintenance.

Reference translation 3:

The US International Airport of Guam and its office has received an email from a self-claimed Arabian millionaire named Laden , which threatens to launch a biochemical attack on such public places as airport . Guam authority has been on alert .

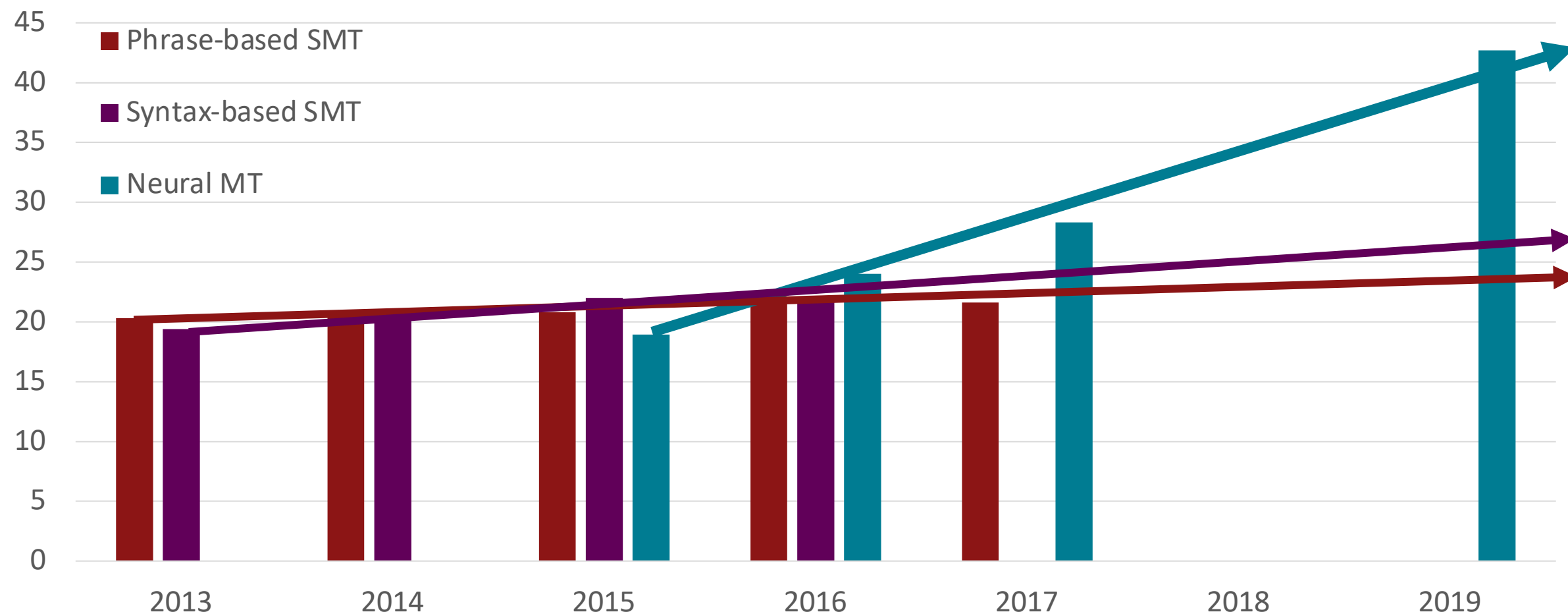
Reference translation 4:

US Guam International Airport and its office received an email from Mr. Bin Laden and other rich businessman from Saudi Arabia . They said there would be biochemistry air raid to Guam Airport and other public places . Guam needs to be in high precaution about this matter .

[Papineni et al. 2002]

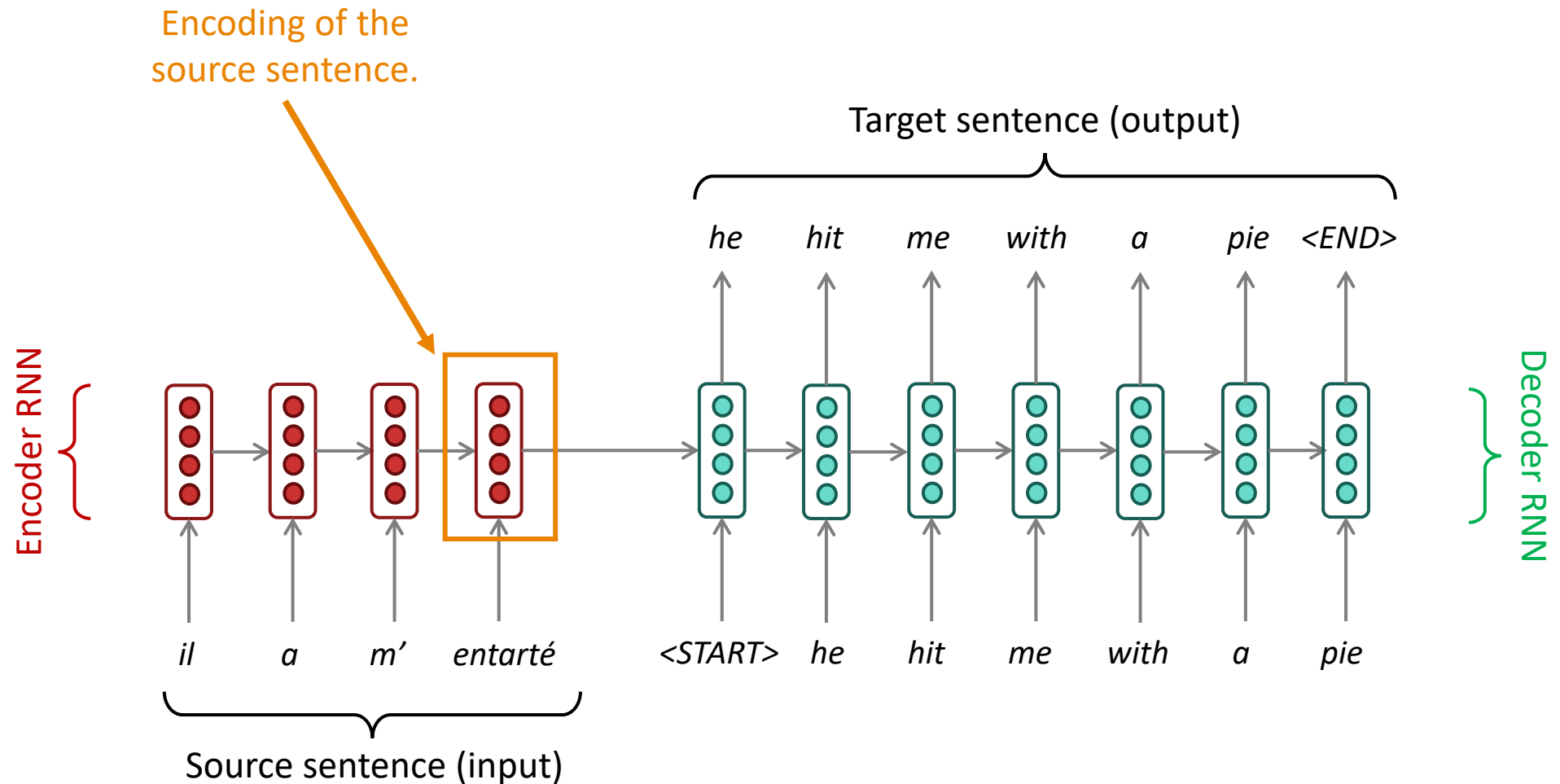
MT progress over time

[Edinburgh En-De WMT newstest2013 Cased BLEU; NMT 2015 from U. Montréal; NMT 2019 FAIR on newstest2019]



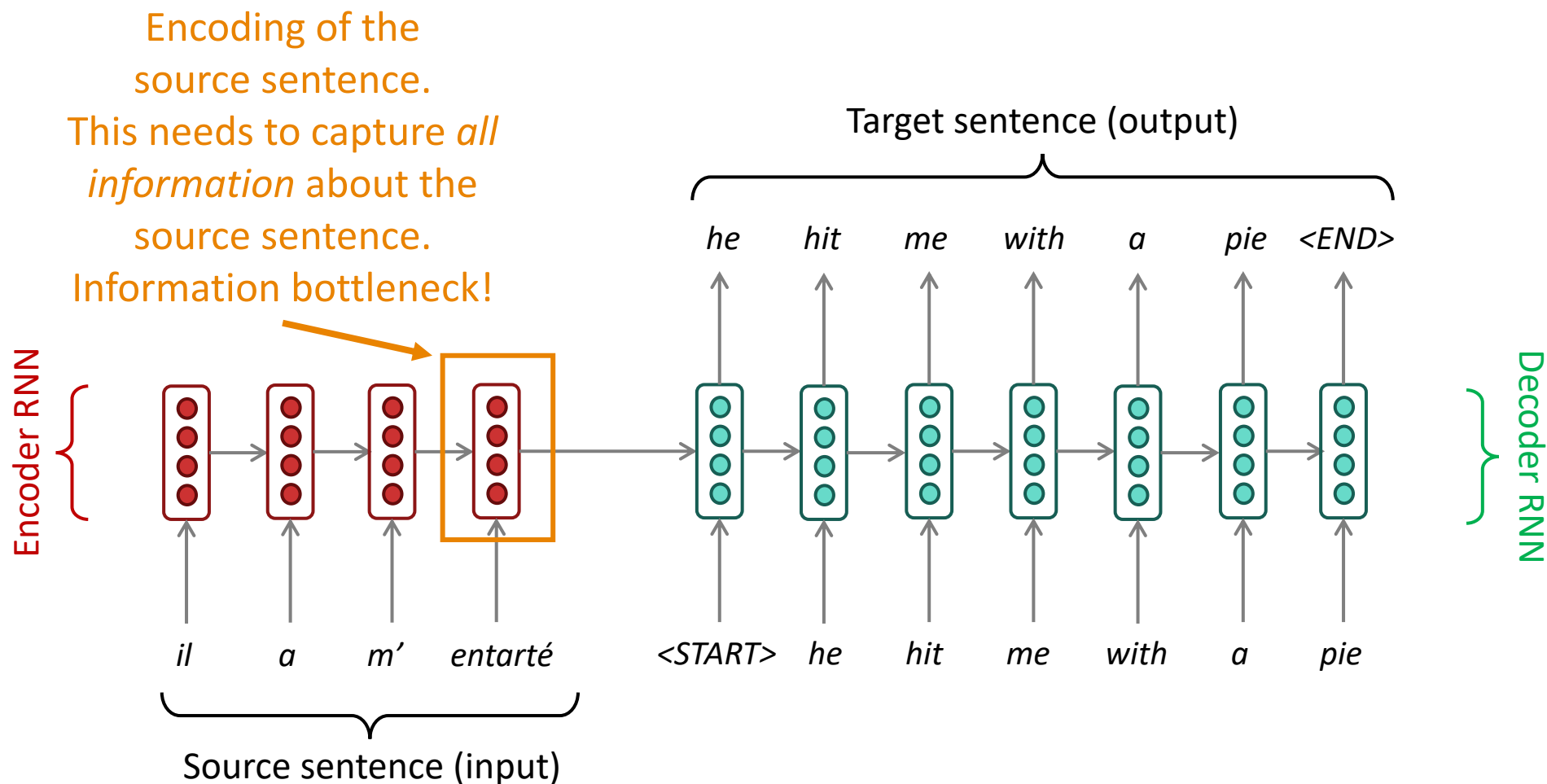
Sources: http://www.meta-net.eu/events/meta-forum-2016/slides/09_sennrich.pdf & <http://matrix.statmt.org/>

2. Why attention? Sequence-to-sequence: the bottleneck problem



Problems with this architecture?

Why attention? Sequence-to-sequence: the bottleneck problem



Attention

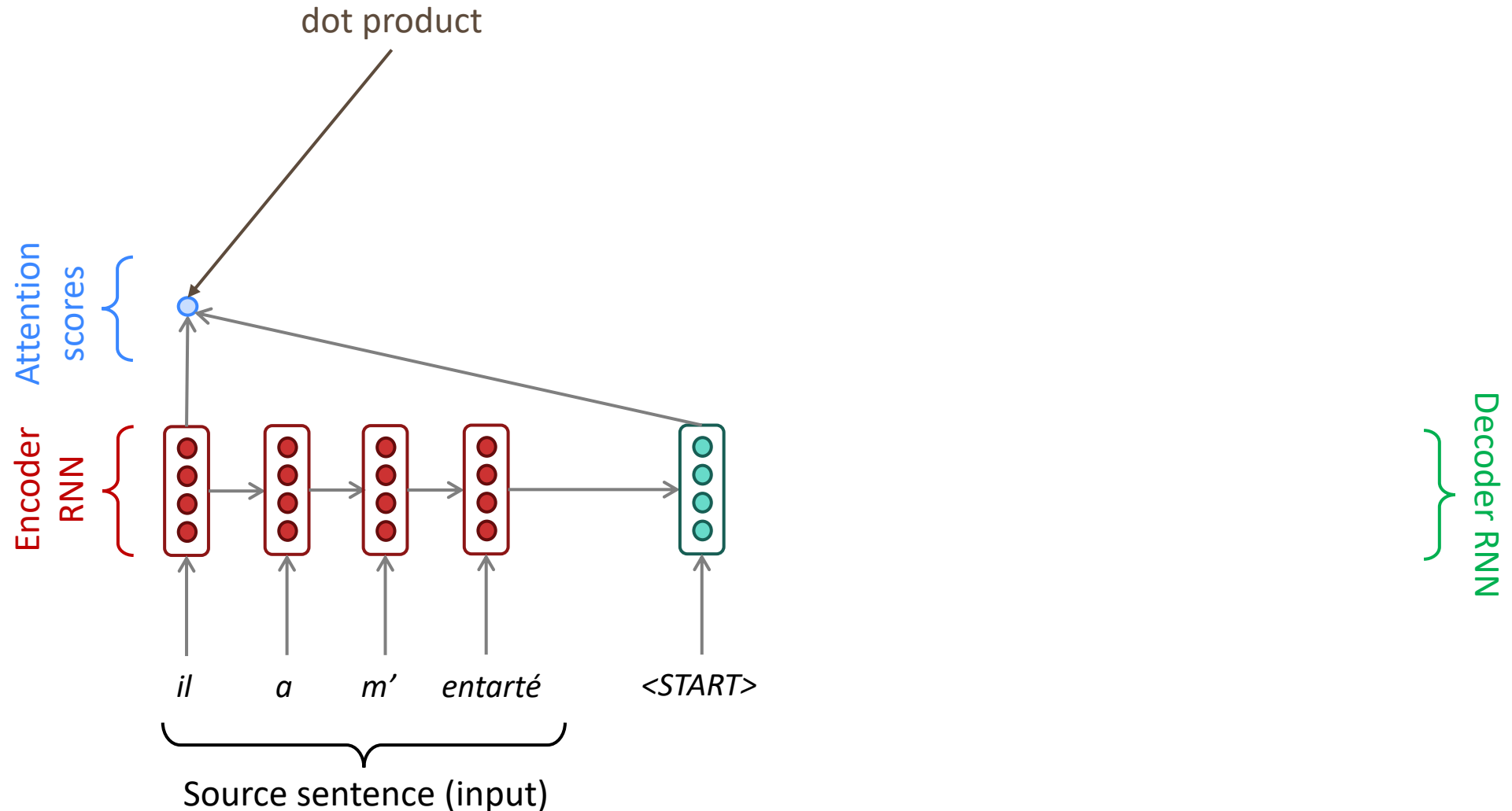
- **Attention** provides a solution to the bottleneck problem.
- **Core idea:** on each step of the decoder, *use direct connection to the encoder to focus on a particular part* of the source sequence



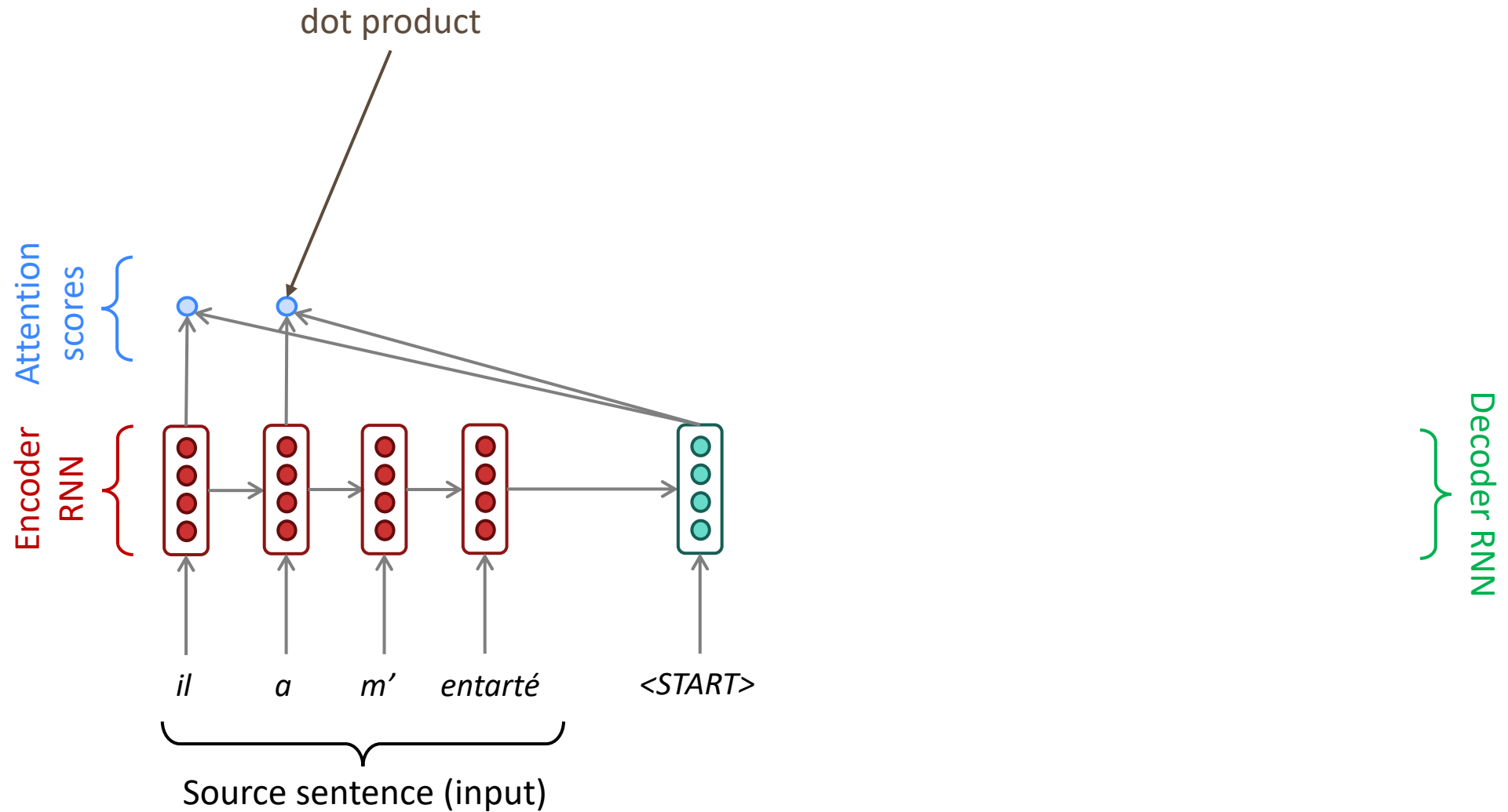
- First, we will show via diagram (no equations), then we will show with equations

Sequence-to-sequence with attention

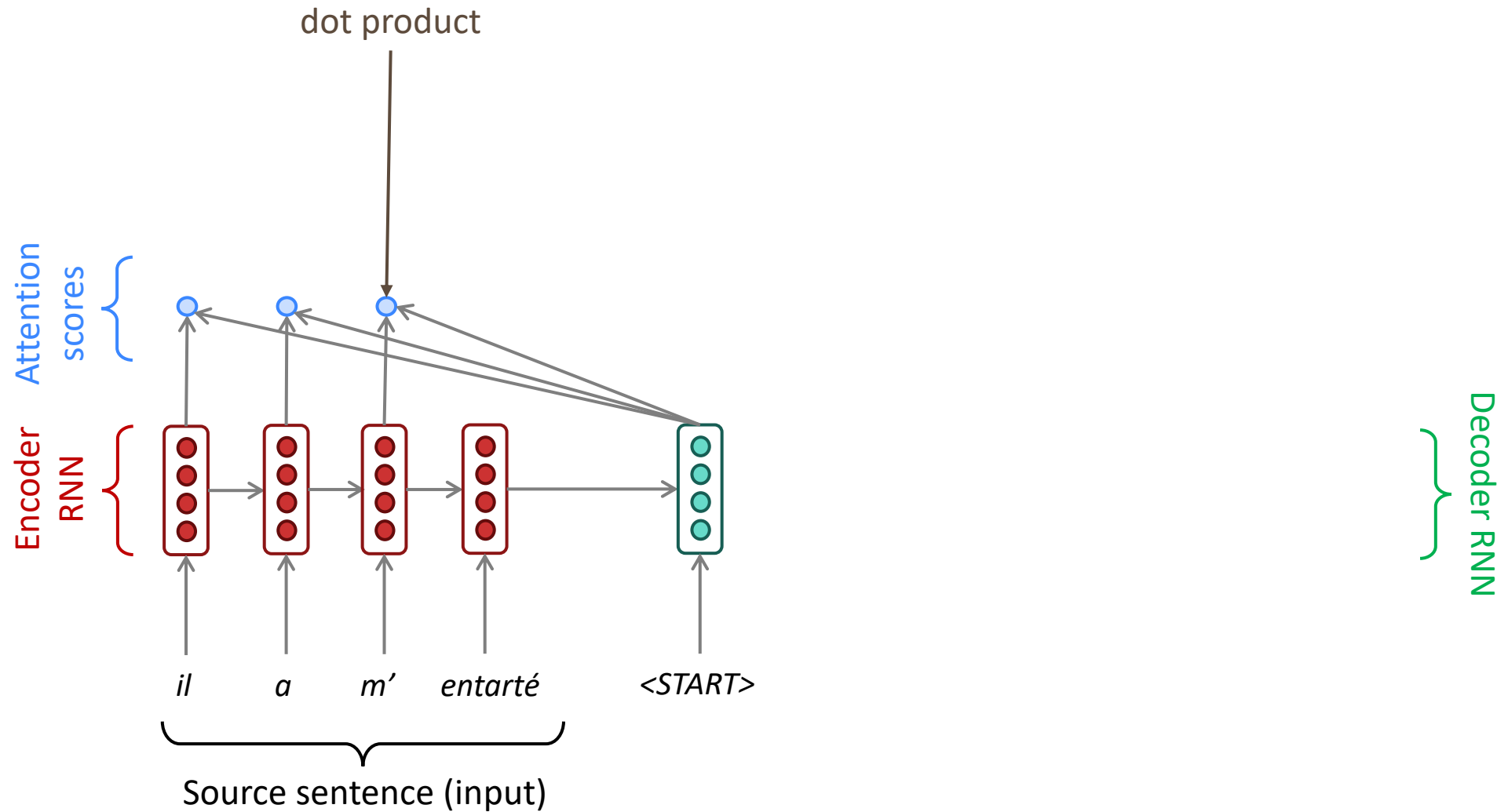
Core idea: on each step of the decoder, *use direct connection to the encoder to focus on a particular part* of the source sequence



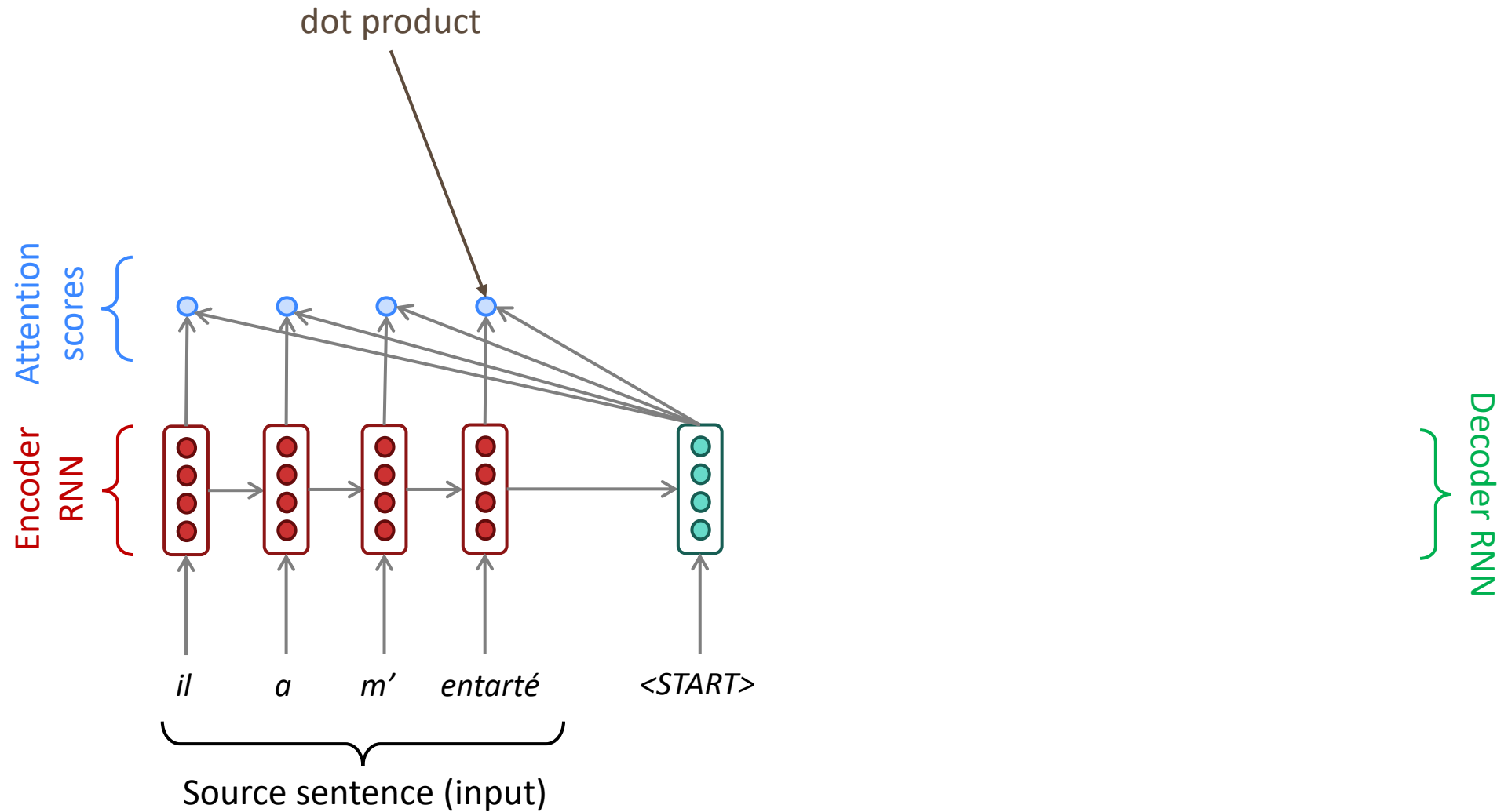
Sequence-to-sequence with attention



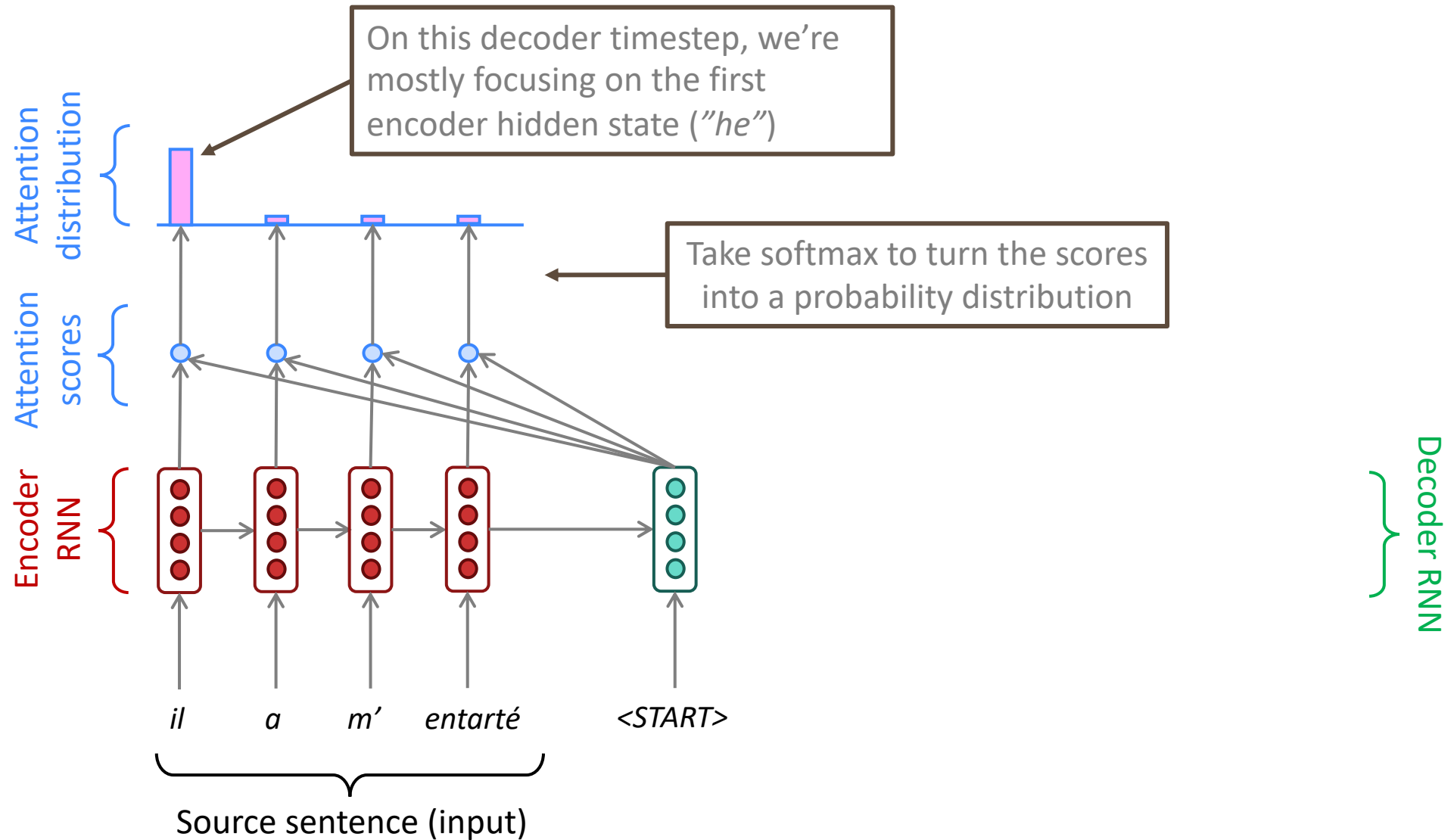
Sequence-to-sequence with attention



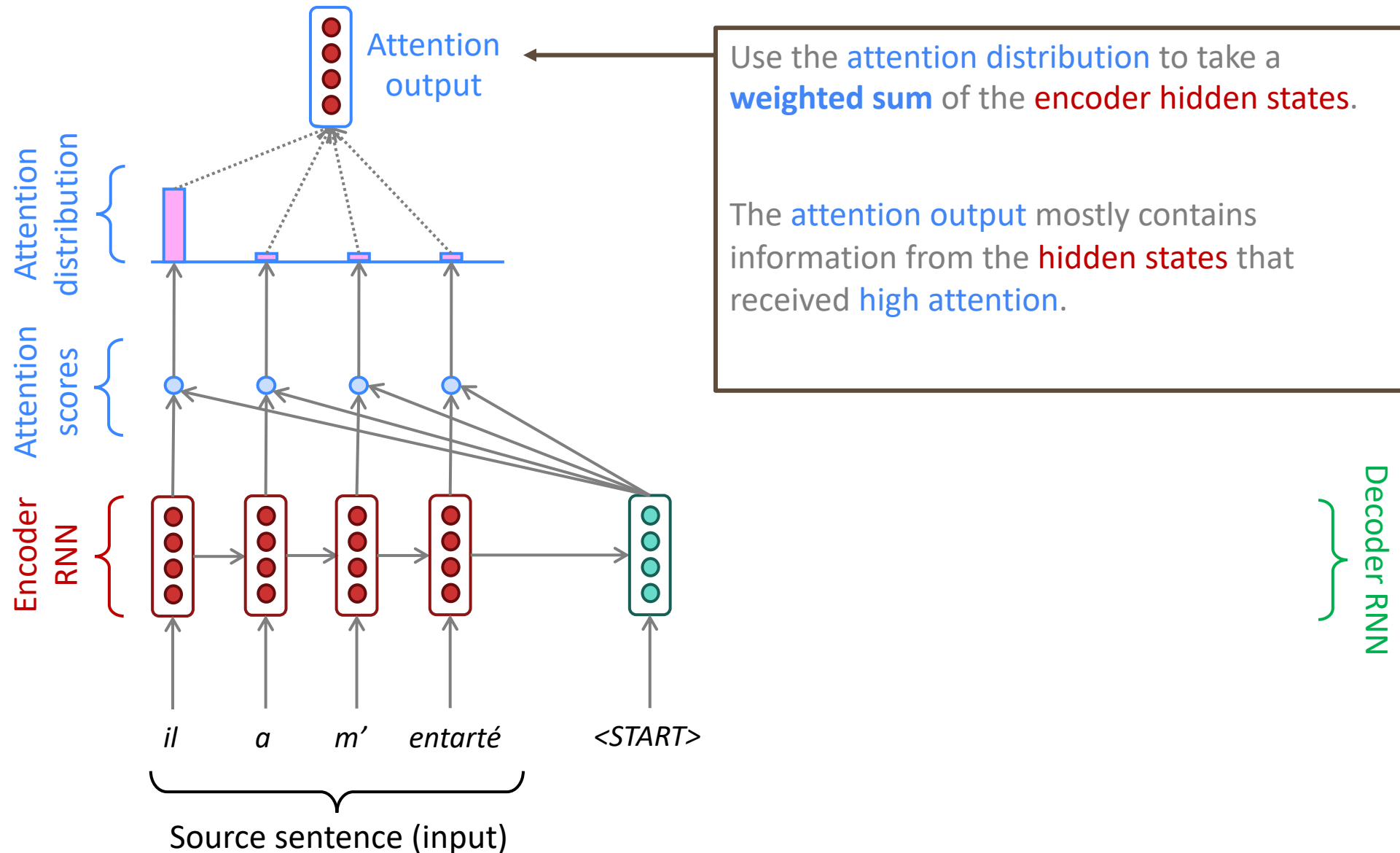
Sequence-to-sequence with attention



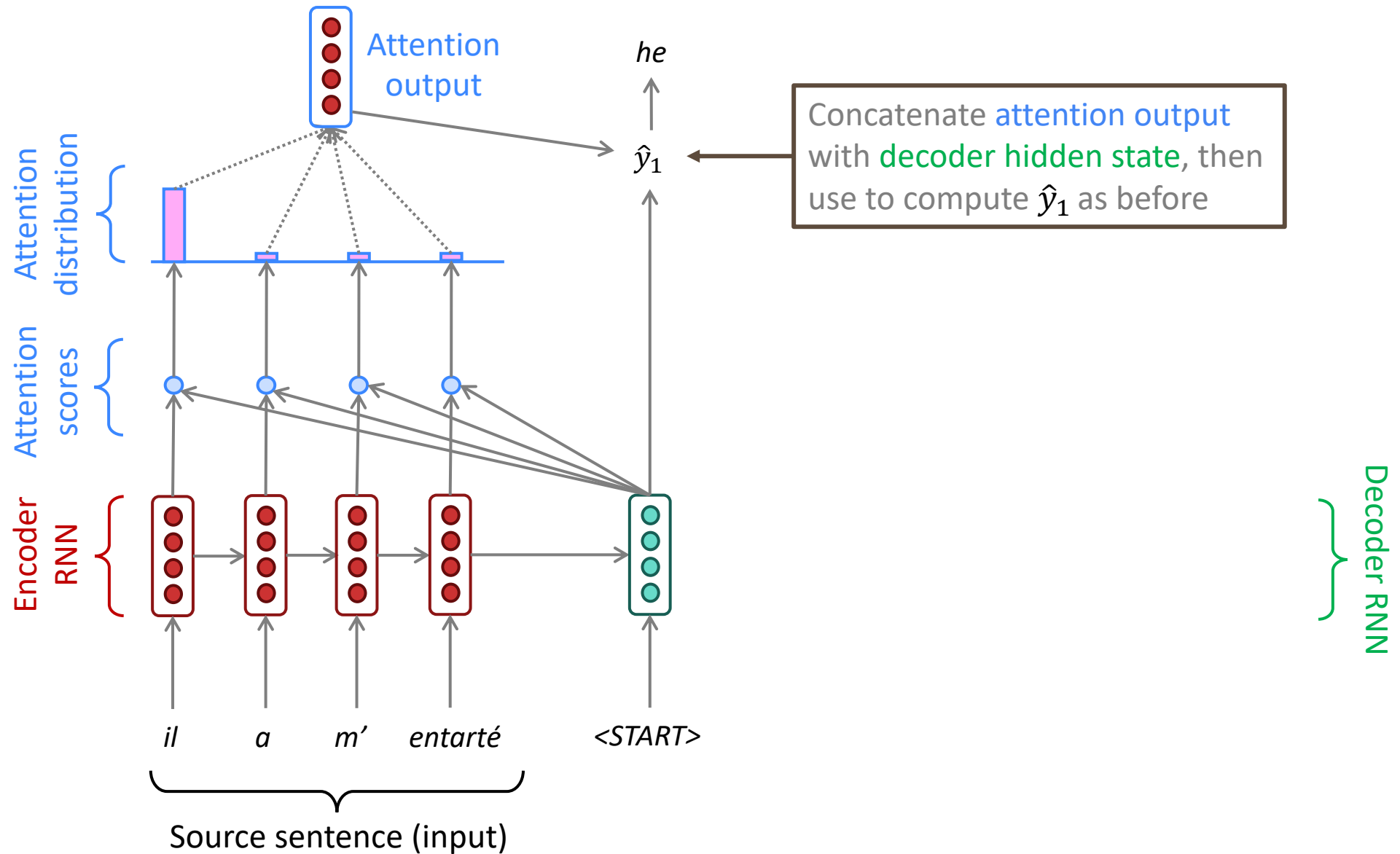
Sequence-to-sequence with attention



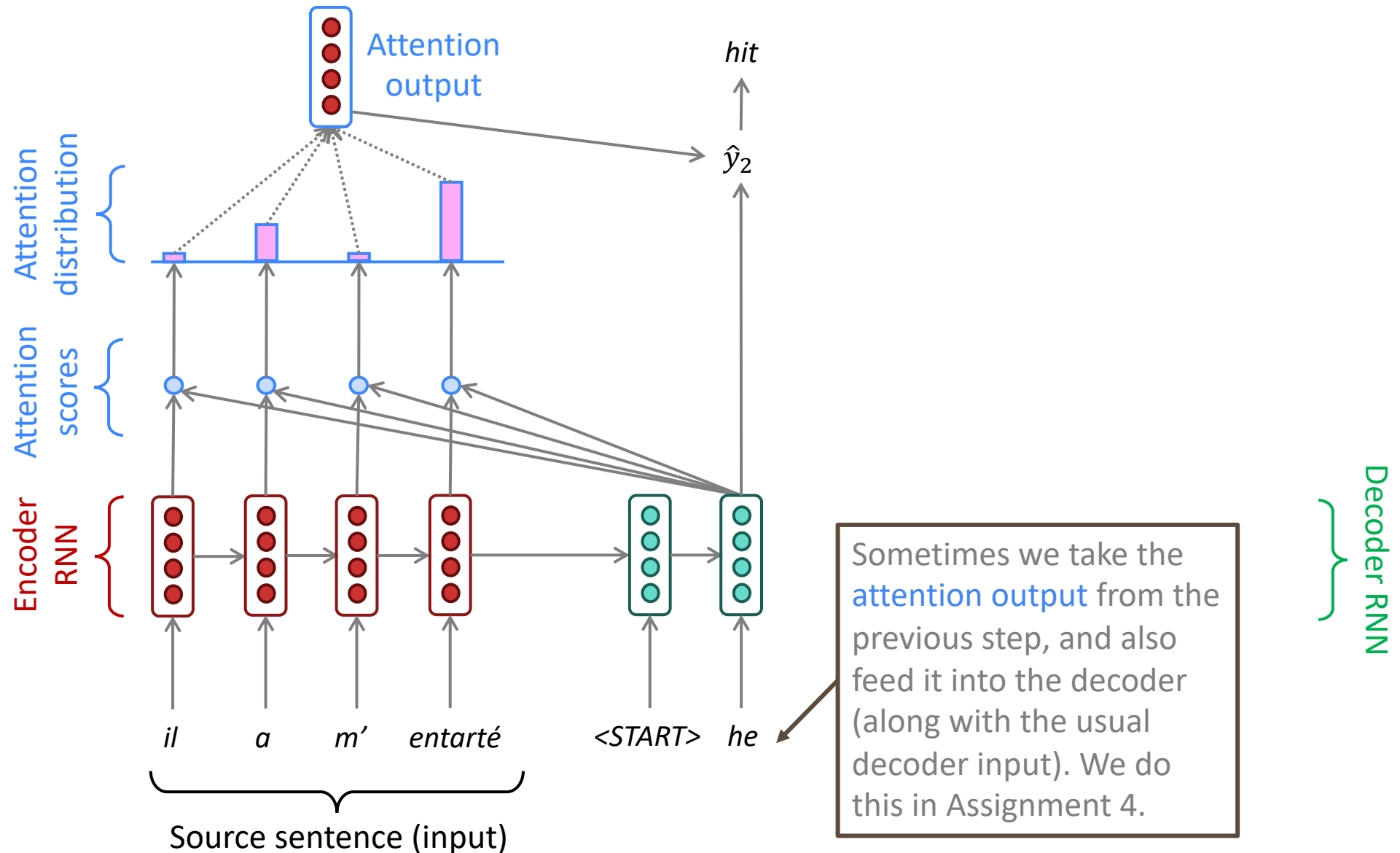
Sequence-to-sequence with attention



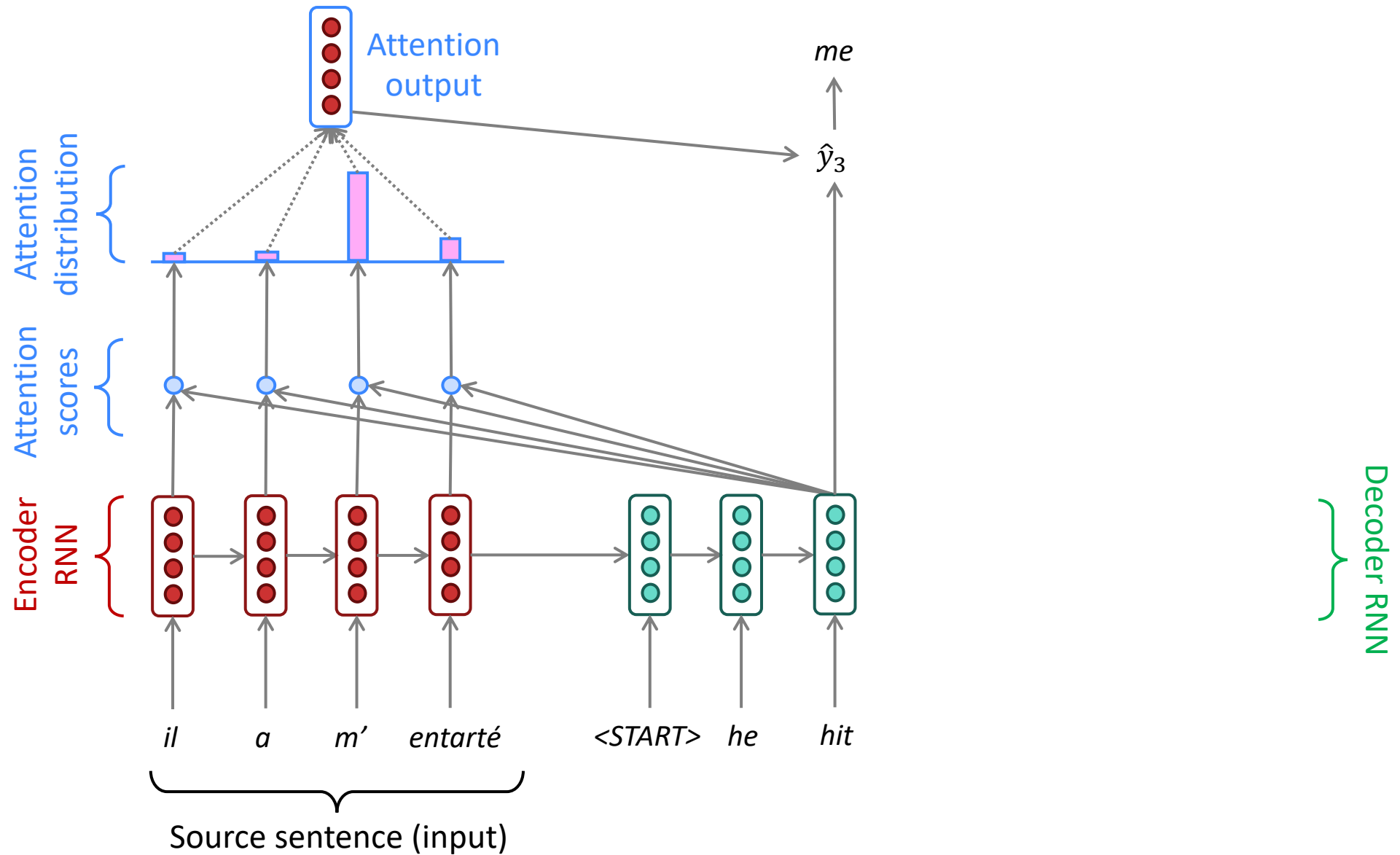
Sequence-to-sequence with attention



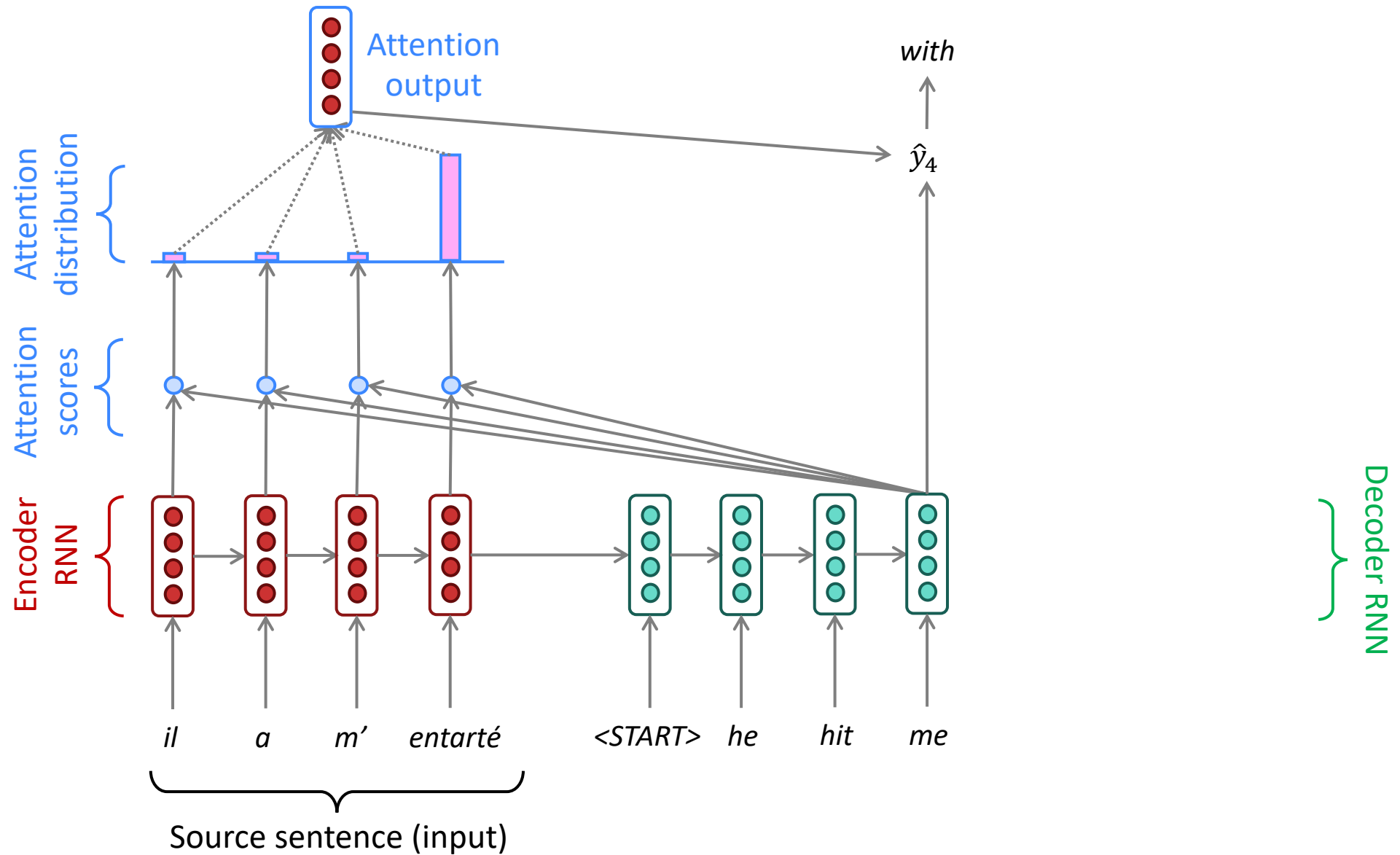
Sequence-to-sequence with attention



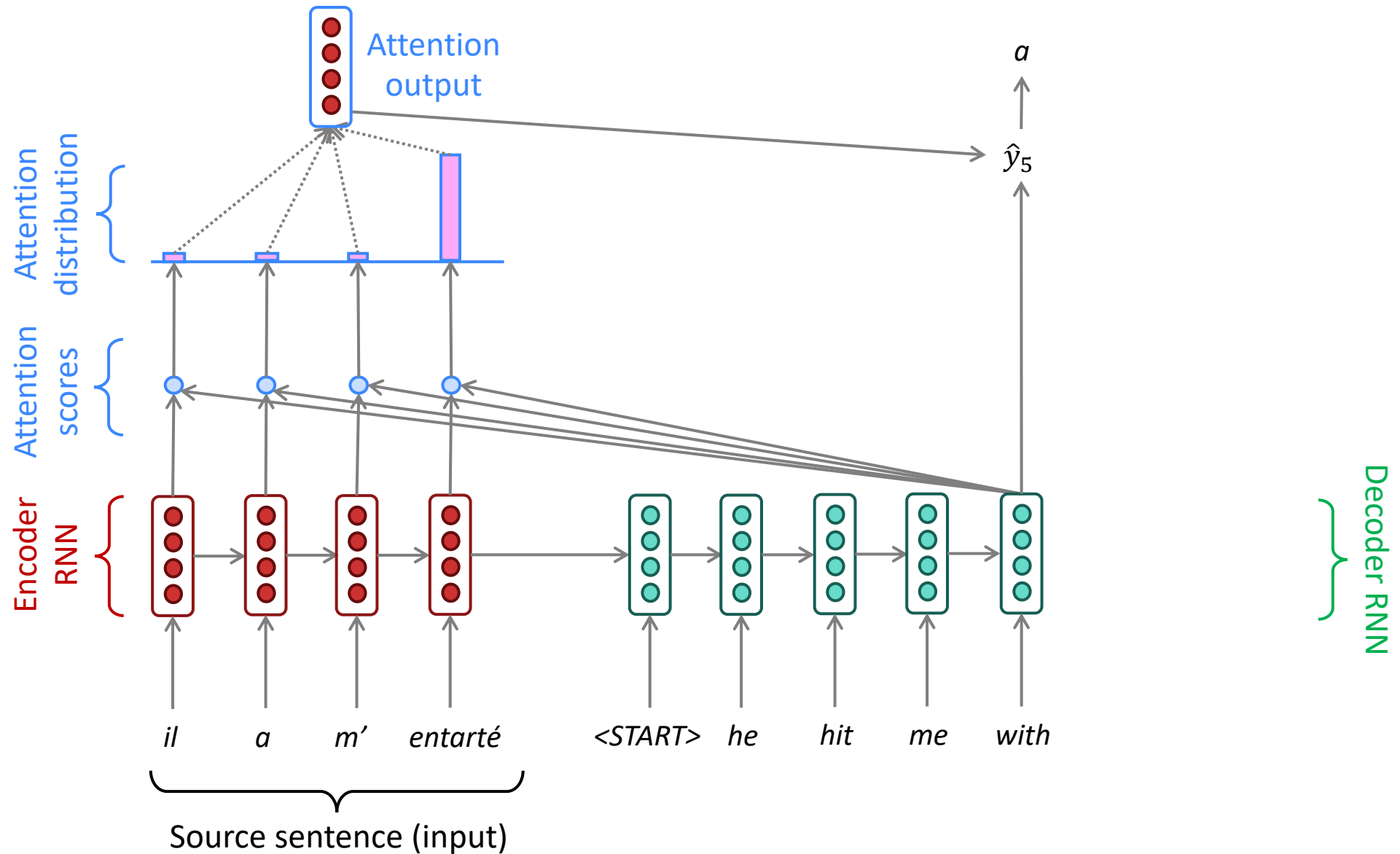
Sequence-to-sequence with attention



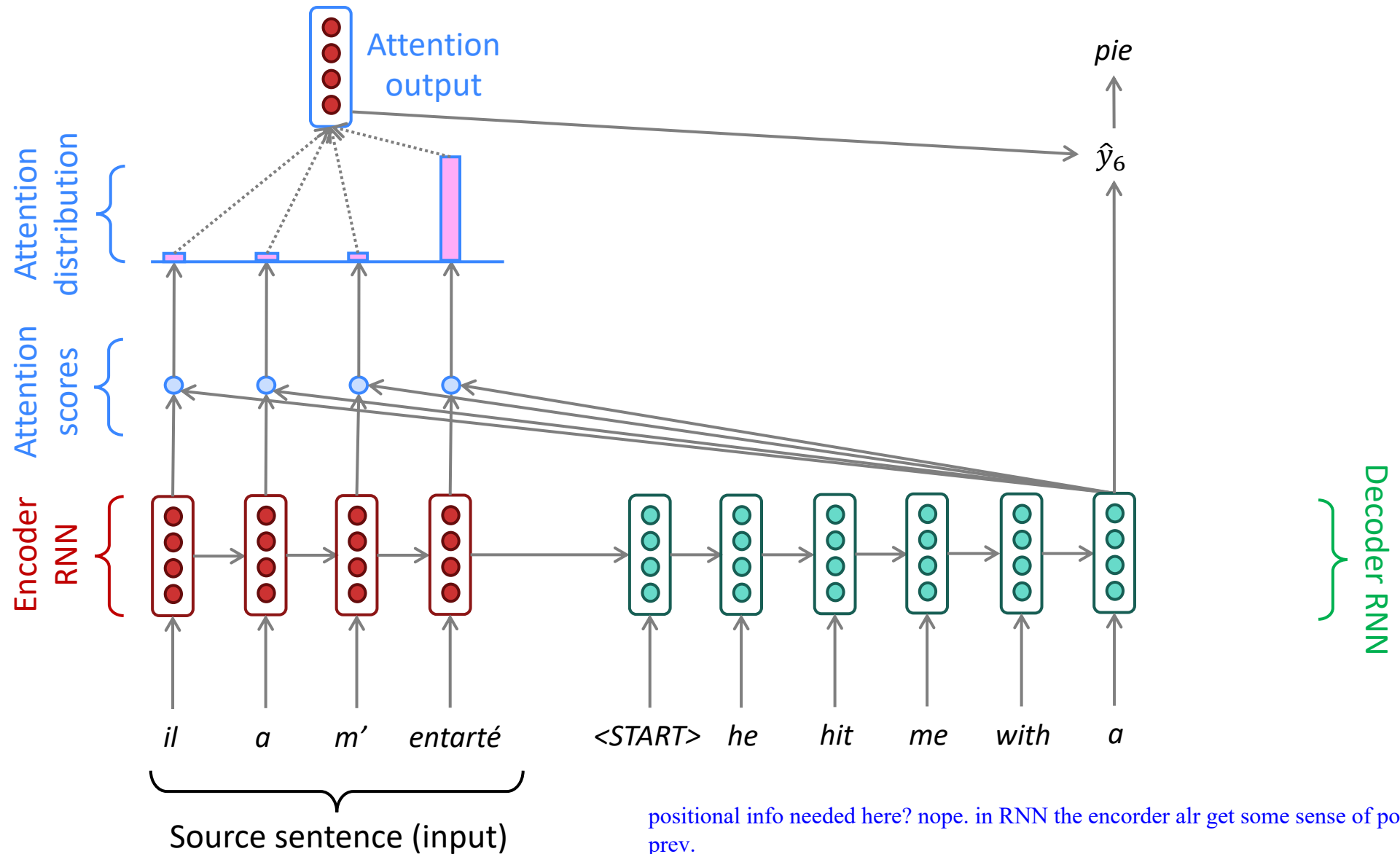
Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention



Attention: in equations

- We have encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$
- On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

dot product of ST with each of the original word's hidden vectors

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution and sums to 1)

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t

$$a_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

$$[a_t; s_t] \in \mathbb{R}^{2h}$$

Attention is great!



- Attention significantly **improves NMT performance**
 - It's very useful to allow decoder to focus on certain parts of the source
- Attention provides a **more “human-like” model** of the MT process
 - You can look back at the source sentence while translating, rather than needing to remember it all
- Attention **solves the bottleneck problem**
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention **helps with the vanishing gradient problem**
 - Provides shortcut to faraway states provide more direct connection to hidden states
- Attention provides **some interpretability**
 - By inspecting attention distribution, we see what the decoder was focusing on
 - We get (soft) **alignment for free!**
 - This is cool because we never explicitly trained an alignment system
 - The network just learned alignment by itself

	he	hit	me	with	a	pie
il						
a						
m'						
entarté						

There are *several* attention variants

- We have some *values* $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^{d_1}$ and a *query* $\mathbf{s} \in \mathbb{R}^{d_2}$

- Attention always involves:

1. Computing the *attention scores* $\mathbf{e} \in \mathbb{R}^N$
2. Taking softmax to get *attention distribution* α :

There are
multiple ways
to do this

$$\alpha = \text{softmax}(\mathbf{e}) \in \mathbb{R}^N$$

3. Using attention distribution to take weighted sum of values:

$$\mathbf{a} = \sum_{i=1}^N \alpha_i \mathbf{h}_i \in \mathbb{R}^{d_1}$$

thus obtaining the *attention output* $\mathbf{a}$ (sometimes called the *context vector*)

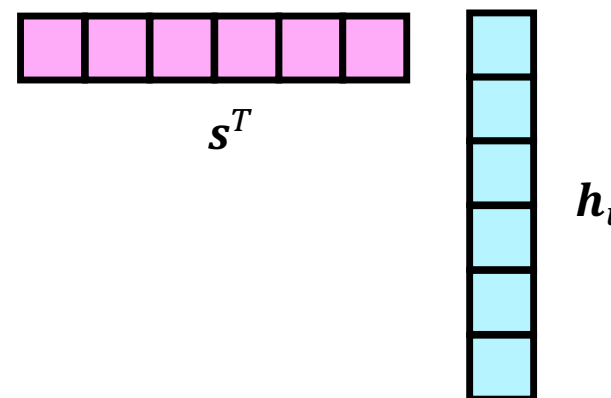
Attention variants

You'll think about the relative advantages/disadvantages of these in Assignment 3!

There are **several ways** you can compute $e \in \mathbb{R}^N$ from $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^{d_1}$ and $\mathbf{s} \in \mathbb{R}^{d_2}$:

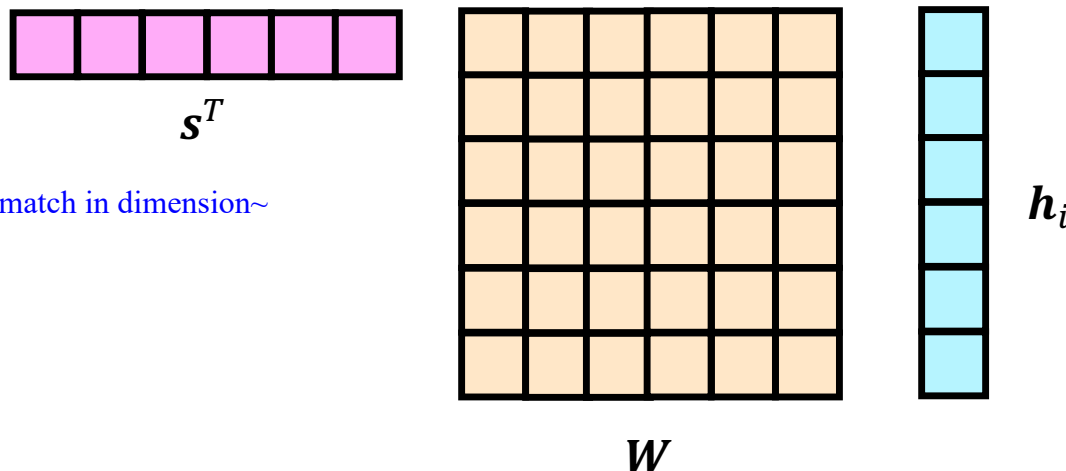
- Basic dot-product attention: $e_i = \mathbf{s}^T \mathbf{h}_i \in \mathbb{R}$

- This assumes $d_1 = d_2$. This is the version we saw earlier.



- Multiplicative attention: $e_i = \mathbf{s}^T \mathbf{W} \mathbf{h}_i \in \mathbb{R}$ [Luong, Pham, and Manning 2015]

- Where $\mathbf{W} \in \mathbb{R}^{d_2 \times d_1}$ is a weight matrix. Perhaps better called “bilinear attention”



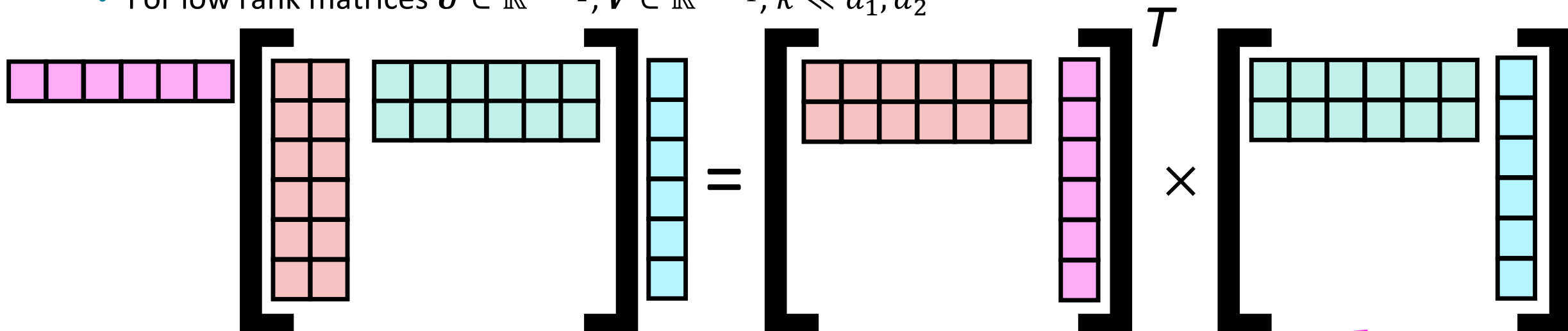
then no longer need $\mathbf{s}^T$ and $\mathbf{h}$ to match in dimension~

Attention variants

You'll think about the relative advantages/disadvantages of these in Assignment 3!

use low rank matrices

- Reduced-rank multiplicative attention: $e_i = s^T (U^T V) h_i = (Us)^T (Vh_i)$
 - For low rank matrices $U \in \mathbb{R}^{k \times d_2}$, $V \in \mathbb{R}^{k \times d_1}$, $k \ll d_1, d_2$



- Additive attention: $e_i = v^T \tanh(W_1 h_i + W_2 s) \in \mathbb{R}$ [Bahdanau, Cho, and Bengio 2014]
 - Where $W_1 \in \mathbb{R}^{d_3 \times d_1}$, $W_2 \in \mathbb{R}^{d_3 \times d_2}$ are weight matrices and $v \in \mathbb{R}^{d_3}$ is a weight vector.
 - d_3 (the attention dimensionality) is a hyperparameter
 - “Additive” is a weird/bad name. It’s really using a feed-forward neural net layer.

Remember this when we look at Transformers next week!

More information: “Deep Learning for NLP Best Practices”, Ruder, 2017. <http://ruder.io/deep-learning-nlp-best-practices/index.html#attention>
“Massive Exploration of Neural Machine Translation Architectures”, Britz et al, 2017, <https://arxiv.org/pdf/1703.03906.pdf>

Attention is a *general* Deep Learning technique

- We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
- However: You can use attention in **many architectures** (not just seq2seq) and **many tasks** (not just MT)
- More general definition of attention:
 - Given a set of vector **values**, and a vector **query**, **attention** is a technique to compute a weighted sum of the values, dependent on the query.
- We sometimes say that the **query attends to the values**.
- For example, in the seq2seq + attention model, each decoder hidden state (query) *attends to* all the encoder hidden states (values).

Attention is a *general* Deep Learning technique

- More general definition of attention:
 - Given a set of vector *values*, and a vector *query*, attention is a technique to compute a weighted sum of the values, dependent on the query.

Intuition:

- The weighted sum is a *selective summary* of the information contained in the values, where the query determines which values to focus on.
- Attention is a way to obtain a *fixed-size representation of an arbitrary set of representations* (the values), dependent on some other representation (the query).

Upshot:

- Attention has become the powerful, flexible, general way pointer and memory manipulation in all deep learning models. A new idea from after 2010! From NMT!

3. Course work and grading policy

- 4 x ~9 day Assignments: 6% + 3 x 14%: 48%
- Final Default or Custom Course Project (1–3 people): 49%
 - Project proposal: 8%; milestone: 6%; poster: 3%; report: 32%
- Participation: 3%
 - Attending guest speaker lectures, Ed, our course evals, karma – see website!
- Late day policy
 - 6 free late days; then 1% of total off per day after that; max 3 late days per assignment
- Collaboration policy: Read the website and the Honor Code!
 - For projects: It's okay to use existing code/resources, but you **must document** it, and
 - You will be graded on your value-add
 - If multi-person: Include a brief statement on the work of each team-mate
 - Almost always, everyone gets the same score, but we reserve the right to differentiate if needed
 - If using project for other classes or as RAs, PhD rotation, etc., you must indicate this in proposal

The Final Project

- For FP, you either
 - Do the default project, which is minBERT and Downstream Tasks
 - Open-ended but an easier start; a good choice for most
 - Propose a custom final project, which we must approve
 - You will receive feedback from a **mentor** (TA/prof/postdoc/PhD)
- You can work in teams of 1–3. Being in a team is encouraged.
 - A larger team project or a project used for multiple classes should be larger and often involves exploring more models or tasks
- You can use any language/framework for your project
 - Though we expect nearly all of you to keep using PyTorch

Custom Final Project

- I'm very happy to talk to people about final projects, but the slight problem is that there's only one of me....
- Look at TA expertise for custom final projects:
 - http://web.stanford.edu/class/cs224n/office_hours.html#staff

Staff information

These are the usual staff members who will be at each session, along with their areas of expertise (which may be useful for finding project advice).
Abbreviations: ML = machine learning, LM = language modeling, RL = reinforcement learning, CV = computer vision.

Day	Time	Room	Staff				
Mon	2:00 PM - 4:00 PM (by Calendly appt. only)	Gates 348	Chris Manning <i>Most areas of NLP. Less good on GANs and RL.</i>				
Mon	2:30 PM - 5:20 PM	160-124	Rashon Poole <i>Chatbot, Human-LLM interaction</i>	Shijia Yang <i>Multimodal LLMs</i>	Ryan Li <i>NLP, Human-AI interaction</i>	Shikhar Murty <i>Language Models, Compositionality, Reasoning, Grounding</i>	
Tue	1:30 PM - 4:20 PM	420-041	Neil Nie <i>CV, LM, VLM, RAG, Robotics</i>	Kaylee Burns <i>Robotics, RL</i>	Zhoujie Ding <i>Trustworthy ML</i>	Jingwen Wu <i>NLP, Healthcare</i>	Josh Singh <i>QA, Conversational Assistants, GNNs</i>
Wed	9:00 AM - 11:50 AM	420-041	Chaofei Fan <i>Brain speech to text</i>	Moussa Doumbouya <i>NLP, Translation, Under-served languages, Computational Education</i>	Archit Sharma <i>Reinforcement learning, RLHF, Robotics</i>	Olivia Lee <i>Language Models, RL, VLMs / Multimodal, Robotics</i>	Yann Dubois <i>LLM finetuning and evaluation, representation learning</i>
Thu	6:00 PM - 8:50 PM	160-B40	Kamyar Salahi <i>Vision and Language</i>	Johnny Chang <i>NLP, RAGs</i>	Soumya Chatterjee <i>NLP, Translation, QA</i>	Aditya Agrawal <i>Multimodal learning, Explainability, GNNs</i>	
Fri	1:30 PM - 4:20 PM	160-B40	Yuan Gao <i>Pretraining, Multimodal LLMs, Generative Models</i>	Anna Goldie <i>Language Models, Reinforcement Learning, Retrieval Augmented Generation (RAG)</i>	Timothy Dai <i>AI for Sustainability</i>	Sonia Chu <i>Vision, time series</i>	
Sat	9:00 AM - 11:50 AM	Shriram 104	Arvind Mahankali <i>Language models, AI for Math, Deep learning theory</i>				

The Default Final Project

- The 2024 final project handouts are on the website now!
- **This year:** Building and experimenting with a minBERT implementation
 - Provided starter code in PyTorch. 😊
- We will discuss transformer models and BERT next in the class (next 2+ lectures)
- What you do:
 - Finish writing an implementation of BERT
 - Fine-tune it for Sentiment analysis
 - Rotten Tomatoes: **Light, silly, photographed with colour and depth, and rather a good time.**
Sentiment: 4 (Positive)
 - Extend and improve it in various ways of your choice:
 - Contrastive learning
 - Paraphrasing
 - Regularized optimization

Why Choose The Default Final Project?

- If you:
 - Have limited experience with research, don't have any clear idea of what you want to do, or want guidance and a goal, ... and a leaderboard, even
- Then:
 - Do the default final project!
 - Many people should do it! (Past statistics: about half of people do DFP.)

Why Choose The Custom Final Project?

- If you:
 - Have some research idea that you're excited about (and are possibly already working on)
 - You want to try to do something different on your own
 - You want to explore more of the process of defining a research goal, finding data and tools, and working out something you could do that is interesting (and perhaps novel), and how to evaluate it
- Then:
 - Do the custom final project!
- **Note: The final project for cs224n must substantively involves both human language and neural networks (the topics of this class)!**

Gamesmanship

- The default final project is more guided, but it should be the same amount of work
- It's just that you can focus on a given problem rather than coming up with your own
- The default final project is also an open-ended project where you can explore different approaches, but to a given problem. Strong default final projects do new things.
- There are great default final projects and great custom final projects ... and there are weak default final projects and weak custom final projects.
 - The path to success is not to do something that looks kinda weak/ill-considered compared to what you could have done with the DFP.
- Neither option is the easy way to a good grade; we give Best Project Awards to both

Computing: How to get GPUs (which you will need)

Confession: We're not in as good a position for providing free compute as previous years

- GCP: Thanks to Google, \$50 credit per person, which can be used for Ass 3, 4, FP
- All clouds (GCP, AWS, Azure, ...): If you haven't used a cloud with an account before, you can usually get some amount of free credit when you begin [Fine point: does it enable GPU use?]
- You can often also usefully use Google Colab, which allows limited GPU use
 - You'll get better GPU availability, especially late in the quarter, if you pay \$10/month for Colab Pro
 - Alternatively, you can use similar Jupyter notebooks in the cloud on:
 - AWS Sagemaker Studio Lab
 - Kaggle Notebooks ... perhaps with better GPU availability for longer
- Modal.com
 - Low price GPU provider; allows a limited amount of free compute per month
 - You can also try other providers like Vast.ai, Lambda Labs, ...

Computing: How to get GPUs (which you will need)

- Together.ai is providing \$50 **API access to LLMs** per FP team if you register with us
- You can use other APIs like OpenAI, Anthropic, etc., but you'll need to pay to use them
 - There are some options for free use of OpenAI models on Microsoft, Perplexity, Gemini on Google, etc., but you usually don't get API use

See the various guides that we have posted on Ed (and more may appear):

- GCP:
<https://docs.google.com/document/d/1FLx0CXIn-SoExxKM1efC-E-6iBjUR4uEnpGnfemMMR0/edit>
- Colab
<https://docs.google.com/document/d/1aNfGnKmdLCMYl8guJ2shGXa-PYCOVCtv5o65Z4GfF4A/edit>
- Modal
https://docs.google.com/document/d/1C1DBeRlAqFyE4yrDctqraOg1RCrUyZ-R9iKqCY_o3_s/edit
- Together
<https://edstem.org/us/courses/57406/discussion/4828063>

Project Proposal – one from every team 8%

1. Find a relevant (key) research paper for your topic
 - For DFP, we provide some suggestions, but you might look elsewhere for interesting work
2. Write a summary of that research paper and what you took away from it as key ideas that you hope to use
3. Write what you plan to work on and how you can innovate in your final project work
 - Suggest a good milestone to have achieved as a halfway point
4. Describe as needed, **especially for Custom projects**:
 - A project plan, relevant existing literature, the kind(s) of models you will use/explore; the **data** you will use (and how it is obtained), and how you will **evaluate** success
5. Write an ethical considerations paragraph, outlining potential ethical challenges of your work if deployed in the real world and practical risk mitigation strategies

3–4 pages, due Thu May 2, 4:30pm on Gradescope (separate CFP and DFP submissions)

Project Proposal – one from every team 8%

2. How to think critically about a research paper for the summary

- What were the main novel contributions or points?
- Is what makes it work something general and reusable or a special case?
- Are there flaws or neat details in what they did?
- How does it fit with other papers on similar topics?
- Does it provoke good questions on further or different things to try?
 - Grading of research paper review is primarily **summative** and is most of the points

3. How to do a good job on your project plan

- You need to have an overall sensible idea (!)
- But most project plans that are lacking are lacking in nuts-and-bolts ways:
 - Do you have appropriate data or a realistic plan to be able to collect it in a short period of time
 - Do you have a realistic way to evaluate your work
 - Do you have appropriate baselines or proposed ablation studies for comparisons
- Grading of project proposal is primarily **formative**

Project Milestone – from everyone 6%

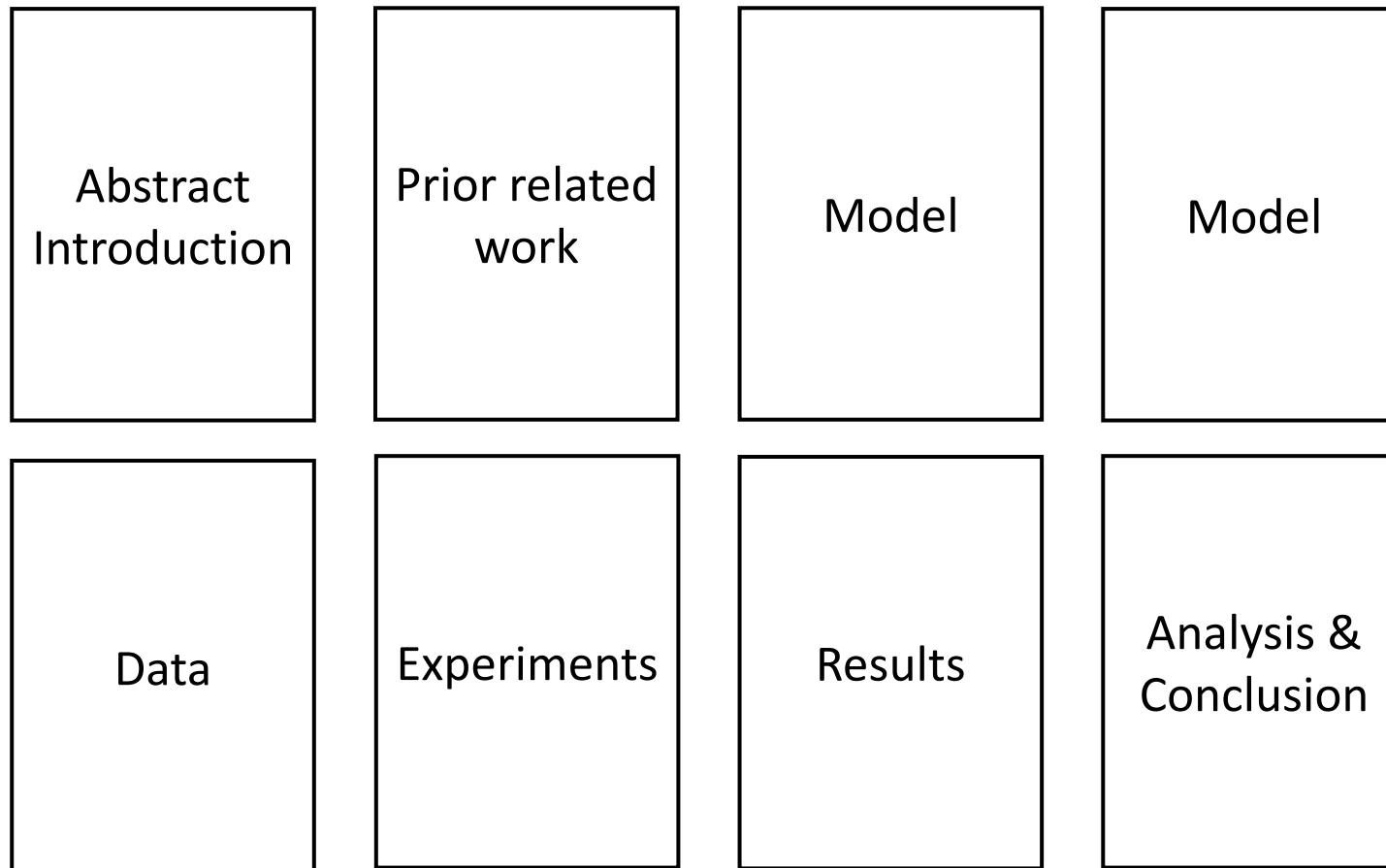
- This is a progress report
- You should be more than halfway done!
- Describe the experiments you have run
- Describe the preliminary results you have obtained
- Describe how you plan to spend the rest of your time

You are expected to **have implemented some system** and to **have some initial experimental results** to show by this date (except for certain unusual kinds of projects)

Due Tue May 21, 4:30pm on Gradescope

Project writeup

- Writeup quality is very important to your grade!!!
 - Look at recent years' prize winners for good examples



Finding Research Topics

Two basic starting points, for all of science:

- [Nails] Start with a (domain) problem of interest and try to find good/better ways to address it than are currently known/used
- [Hammers] Start with a technical method/approach of interest, and work out good ways to extend it, improve it, understand it, or find new ways to apply it

Project types

This is not an exhaustive list, but most projects are one of

1. Find an application/task of interest and explore how to approach/solve it effectively, often with an existing model
 - Could be a task in the wild or, more commonly, some existing Kaggle/bake-off/shared task
2. Implement a complex neural architecture (perhaps novel) and explore its performance on some data
3. Explore using prompting (in-context learning) or language model programs
 - The above types are expected to have experiments with numbers and ablations 😊
4. Analysis/Interpretability project. Analyze the behavior of a model: how it represents linguistic or world knowledge, or what kinds of phenomena it can handle or errors that it makes
5. Rare theoretical or linguistic project: Show some interesting, non-trivial properties of a model type, data, or a data representation

Deep Poetry: Word-Level and Character-Level Language Models for Shakespearean Sonnet Generation

Stanley Xie, Ruchir Rastogi and Max Chang

Gated LSTM

Thy youth 's time and face his form shall cover?
Now all fresh beauty, my love there
Will ever Time to greet, forget each, like ever decease,
But in a best at worship his glory die.

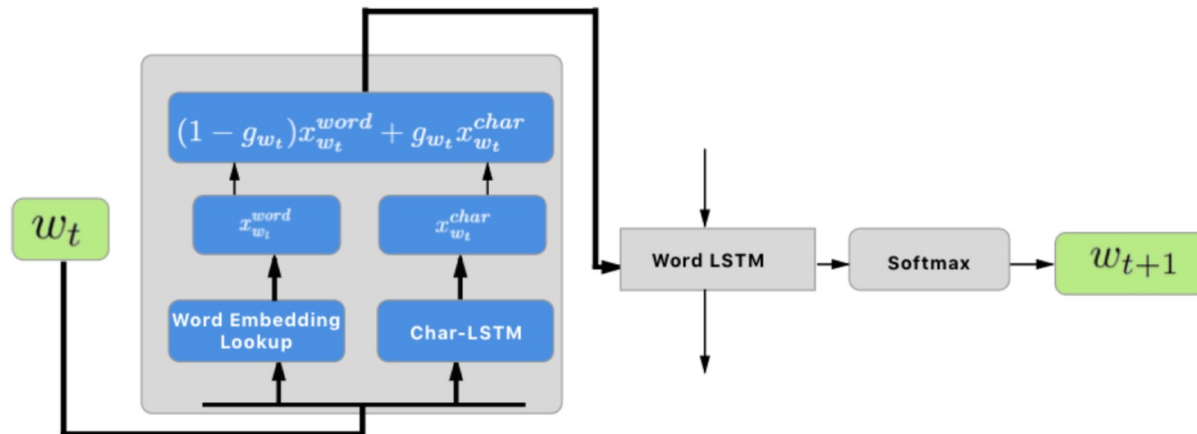


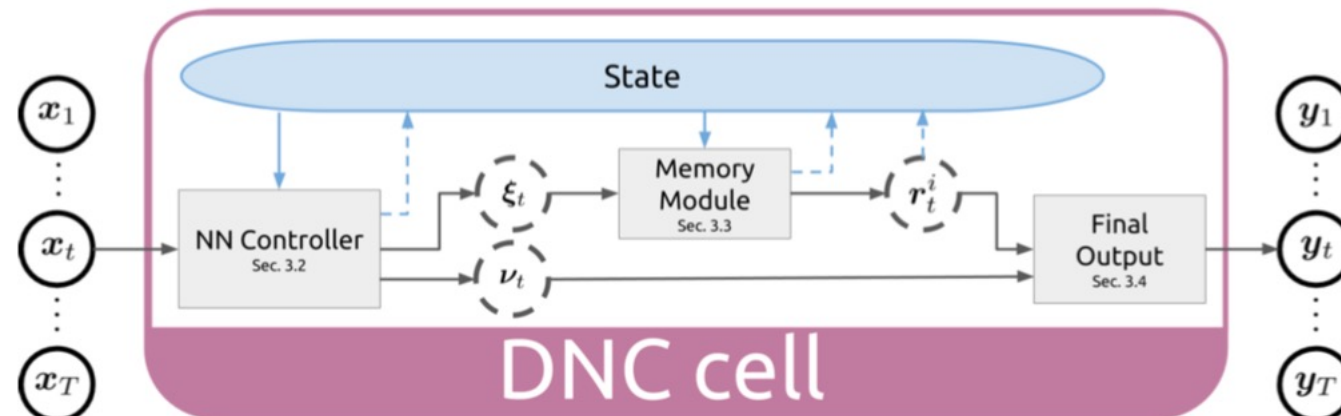
Figure 1: Architecture of the Gated LSTM

Implementation and Optimization of Differentiable Neural Computers

Carol Hsin

Graduate Student in Computational & Mathematical Engineering

We implemented and optimized Differentiable Neural Computers (DNCs) as described in the Oct. 2016 DNC paper [1] on the bAbI dataset [25] and on copy tasks that were described in the Neural Turing Machine paper [12]. This paper will give the reader a better understanding of this new and promising architecture through the documentation of the approach in our DNC implementation and our experience of the challenges of optimizing DNCs.



Improved Learning through Augmenting the Loss

Hakan Inan

inanh@stanford.edu

Khashayar Khosravi

khosravi@stanford.edu

We present two improvements to the well-known Recurrent Neural Network Language Models(RNNLM). First, we use the word embedding matrix to project the RNN output onto the output space and already achieve a large reduction in the number of free parameters while still improving performance. Second, instead of merely minimizing the standard cross entropy loss between the prediction distribution and the "one-hot" target distribution, we minimize an additional loss term which takes into account the inherent metric similarity between the target word and other words. We show with experiments on the Penn Treebank Dataset that our proposed model (1) achieves significantly lower average word perplexity than previous models with the same network size and (2) achieves the new state of the art by using much fewer parameters than used in the previous best work.

Published as a conference paper at ICLR 2017

Enhancing Self-Consistency and Performance of Pre-Trained Language Models through Natural Language Inference

Eric Mitchell, Joseph J. Noh, Siyan Li, William S. Armstrong,
Ananth Agarwal, Patrick Liu, Chelsea Finn, Christopher D. Manning
Stanford University
eric.mitchell@cs.stanford.edu

Abstract

While large pre-trained language models are powerful, their predictions often lack logical consistency across test inputs. For example, a state-of-the-art Macaw question-answering (QA) model answers *Yes* to *Is a sparrow a bird?* and *Does a bird have feet?* but answers *No* to *Does a sparrow have feet?*. To address this failure mode, we propose a framework, Consistency Correction through Relation Detection, or **ConCoRD**, for boosting the consistency and accuracy of pre-trained NLP models using pre-trained natural language inference (NLI) models without fine-tuning or re-training. Given a batch of test inputs, ConCoRD samples several candidate outputs for each input and instantiates a factor graph that accounts for both the model's belief about the likelihood of each answer choice in isolation and the NLI model's beliefs about pair-wise answer choice compatibility. We show that a weighted MaxSAT solver can efficiently compute high-

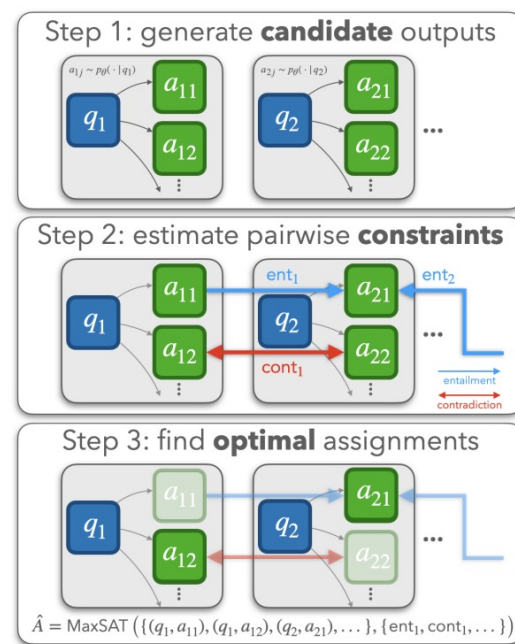


Figure 1: ConCoRD first generates candidate outputs from the base pre-trained model, then estimates soft pairwise constraints between output choices, and finally finds the most satisfactory choices of answers accounting for both the base model and NLI model's beliefs.

Word2Bits - Quantized Word Vectors

Maximilian Lam

maxlam@stanford.edu

Abstract

Word vectors require significant amounts of memory and storage, posing issues to resource limited devices like mobile phones and GPUs. We show that high quality quantized word vectors using 1-2 bits per parameter can be learned by introducing a quantization function into Word2Vec. We furthermore show that training with the quantization function acts as a regularizer. We train word vectors on English Wikipedia (2017) and evaluate them on standard word similarity and analogy tasks and on question answering (SQuAD). Our quantized word vectors not only take 8-16x less space than full precision (32 bit) word vectors but also outperform them on word similarity tasks and question answering.

Fine-tuning CodeLlama-7B on Synthetic Training Data for Fortran Code Generation using PEFT

Stanford CS224N Custom Project

Soham Govande

Department of Computer Science
Stanford University
govande@stanford.edu

Taeuk Kang

Department of Computer Science
Stanford University
taeuk@stanford.edu

Andrew Shi

Department of Computer Science
Stanford University
acshi@stanford.edu

Table 1: Code Quality Comparison on Fine-Tuned Model and CodeLlama-7B-Instruct

Model	Function Written	Compiled	Executed
Fine-Tuned Model	522	185	125
CodeLlama-7B-Instruct	448	131	50
	Output Generated	Timed Out	Runtime Error
Fine-Tuned Model	121	2	2
CodeLlama-7B-Instruct	49	0	1
	Incorrect	Partially Correct	Correct
Fine-Tuned Model	32	33	56
CodeLlama-7B-Instruct	4	13	32

Winter 2024 project

AI-Driven Fashion Cataloging: Transforming Images into Textual Descriptions

Stanford CS224N Custom Project

SiYi Ma
Department of Computer Science
Stanford University
siyima00@stanford.edu

Nishant Gopinath
Department of Computer Science
Stanford University
nishgop@stanford.edu

Product Image



Ground Truth

```
{  
  "category": "dress",  
  "silhouette": "sheath",  
  "fitting": "loose",  
  "pattern": "floral",  
  "shoulder-style": "sleeveless",  
  "neckline": "high",  
  "length": "mini",  
  "sleeve-length": "long-sleeves"  
}
```

Pre-Trained cogVLM

```
{  
  "category": "dress",  
  "silhouette": "a-line",  
  "fitting": "fitted",  
  "pattern": "floral",  
  "shoulder-style": "sleeveless",  
  "neckline": "high",  
  "length": "mini",  
  "sleeve-length": "long-sleeves"  
}
```

Fine-Tuned cogVLM

```
{  
  "category": "dress",  
  "silhouette": "sheath",  
  "fitting": "loose",  
  "pattern": "floral",  
  "shoulder-style": "sleeveless",  
  "neckline": "high",  
  "length": "mini",  
  "sleeve-length": "long-sleeves"  
}
```

Enhanced Accuracy through Fine-Tuning

Winter 2024 project

4. How to find an interesting place to start?

- Look at ACL anthology for NLP papers:
 - <https://aclanthology.org/>
- Also look at the online proceedings of major ML conferences:
 - NeurIPS <https://papers.nips.cc>, ICML, ICLR <https://openreview.net/group?id=ICLR.cc>
- Look at past cs224n projects
 - <http://cs224n.stanford.edu/>
- Look at online preprint servers, especially:
 - <https://arxiv.org>
- Even better: look for an interesting problem in the world!
 - Hal Varian: How to Build an Economic Model in Your Spare Time
<https://people.ischool.berkeley.edu/~hal/Papers/how.pdf>

Want to try to beat the state of the art on something?

Some sites that try to collate info on the state of the art

- Not always correct, though

<https://paperswithcode.com/sota>

<https://nlpprogress.com/>

Specific tasks/topics. Many, e.g.:

<https://gluebenchmark.com/leaderboard/>

<https://www.conll.org/previous-tasks/>

wse > Natural Language Processing > Machine Translation



Machine Translation

223 papers with code · Natural Language Processing

Machine translation is the task of translating a sentence in a source language to a different language.

State-of-the-art leaderboards

Dataset	Best Method	Paper title	Paper	Code
WMT2014 English-French	Transformer Big + BT	Understanding Back-Translation at Scale		
WMT2014 English-German	Transformer Big + BT	Understanding Back-Translation at Scale		
IWSLT2015 German-English	Transformer	Attention Is All You Need		
WMT2016 English-Romanian	ConvS2S BPE40k	Convolutional Sequence to Sequence Learning		

Finding a topic

- Turing award winner and Stanford CS emeritus professor Ed Feigenbaum says to follow the advice of his advisor, AI pioneer, and Turing and Nobel prize winner Herb Simon:
 - “If you see a research area where many people are working, go somewhere else.”
- But where to go? Wayne Gretzky:
 - “I skate to where the puck is going, not where it has been.”

Old Deep Learning (NLP), new Deep Learning NLP

- In the early days of the Deep Learning revival (2010-2018), most of the work was in defining and exploring better deep learning architectures
- Typical paper:
 - I can improve a summarization system by not only using attention standardly, but allowing copying attention – where you use additional attention calculations and an additional probabilistic gate to simply copy a word from the input to the output
- That's what a lot of good CS 224N projects did too
- In 2019–2024, that approach is dead
 - Well, that's too strong, but it's difficult and much rarer
- Most work downloads a big pre-trained model (which fixes the architecture)
 - Action is in fine-tuning, or domain adaptation followed by fine-tuning, etc., etc.

2024 NLP ... recommended for most of your practical projects 😊

```
pip install transformers # By Huggingface 🙌 Tutorial: Fri May 3, TBD  
# not quite runnable code but gives the general idea....  
from transformers import BertForSequenceClassification, AutoTokenizer  
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')  
model.train()  
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')  
fine_tuner = Trainer( model=model, args=training_args, train_dataset=train_dataset,  
    eval_dataset=test_dataset )  
fine_tuner.train()  
eval_dataset = load_and_cache_examples(args, eval_task, tokenizer, evaluate=True)  
results = evaluate(model, tokenizer, eval_dataset, args)
```

Exciting areas 2024

A lot of what is exciting now is problems that work within or around this world

- Evaluating and improving models for something other than accuracy
 - Evaluating the robustness of models; adaptation when there is domain shift
- Doing empirical work looking at what large pre-trained models have learned & how
- Working out how to get knowledge and good task performance from large models for particular tasks without much data (in context learning, transfer learning, etc.)
- Looking at the bias, trustworthiness, and explainability of large models
- Working on how to augment the data for models to improve performance
- Looking at low resource languages or problems
- Improving performance on the tail of rare stuff, addressing bias
- Building larger language model programs
 - Look at RAG (Retrieval Augmented Generation), langchain, DSPy from Stanford!

Exciting areas 2024

- Scaling models up and down
 - Building big models is BIG ... **but just not possible for a cs224n project**
 - Do think and be realistic about the scale of compute you can do!
 - Building small, performant models is also BIG. This could be a great project
 - Model pruning or model quantization, e.g.: <https://arxiv.org/pdf/2004.07320.pdf>
 - How well can you do QA in 6GB or 500MB? <https://efficientqa.github.io>
 - Doing parameter-efficient fine tuning and low-rank adaptation, etc.
- Looking to achieve more advanced functionalities
 - E.g., compositionality, systematic generalization, fast learning (e.g., meta-learning) on smaller problems and amounts of data, and more quickly
 - COGS: <https://github.com/najoungkim/COGS> or ReCOGS: <https://github.com/frankaging/ReCOGS>
 - gSCAN: <https://arxiv.org/abs/2003.05161>

Can I use ChatGPT (GPT-4) or Gemini etc. to do my final project?

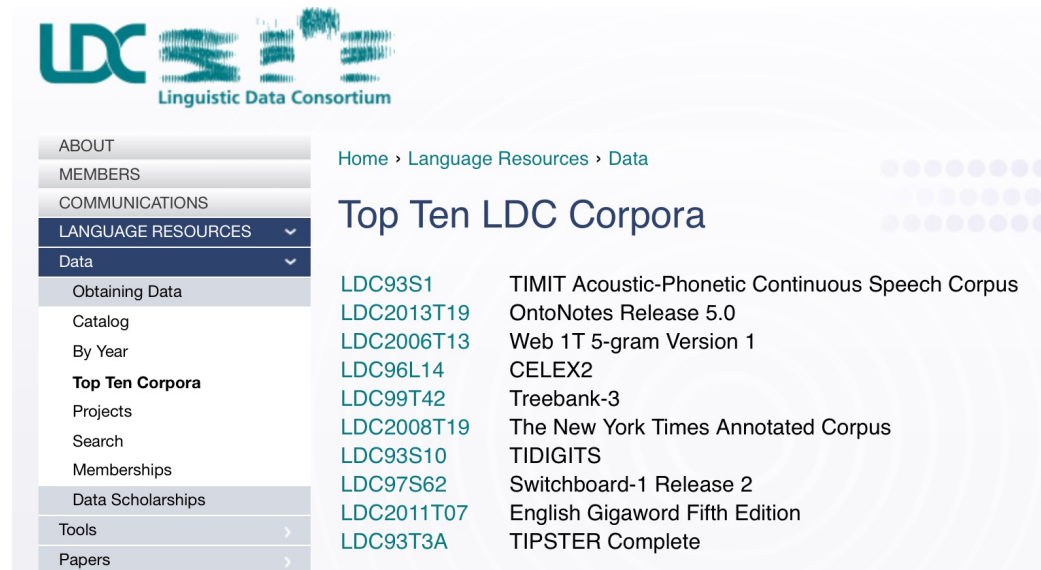
- You need to be very cognizant of how large a model you can train
 - You just don't have the resources to train your own GPT-2 model. Let alone Llama.
 - You probably don't have the resources to even **load** T5 11B
- You are welcome to use ChatGPT, Claude, Gemini, etc. in your final project
 - We can make some API access available for students via gift from Together.ai
- There are almost certainly interesting projects that you can do using them
 - Analysis projects
 - Learning good prompts
 - Chain of thought reasoning
 - Larger language model systems
- But be careful to remember that you will be evaluated based on what you have done – and not on the amazing ChatGPT output that you show us (“look, it works zero shot”)

5. Finding data for your projects

- A few people collect their own data – **we like that, but it's tricky to do quickly enough!**
 - You may have a project that uses “unsupervised” data
 - You can annotate a small amount of data
 - You can find a website that effectively provides annotations, such as likes, stars, ratings, responses, etc.
 - This let's you learn about real word challenges of applying ML/NLP!
 - **But be careful on scoping things so that this doesn't take most of your time!!!**
- Some people have existing data from a research project or company
 - Fine to use providing you can provide data samples for submission, report, etc.
- **Most people make use of an existing, curated dataset built by previous researchers**
 - You get a fast start and there is obvious prior work and baselines

Traditional Linguistic Data Provider: Linguistic Data Consortium

- <https://catalog.ldc.upenn.edu/>
- Stanford licenses this data; you can get access. Sign up/ask questions at: <https://linguistics.stanford.edu/resources/resources-corpora>
- Treebanks, named entities, coreference data, lots of clean newswire text, lots of speech with transcription, parallel MT data, etc.
 - Look at their catalog
 - Don't use for non-Stanford purposes!





Huggingface Datasets

- <https://huggingface.co/datasets>

Hugging Face

Search models, datasets, users...

Models

Datasets

Pricing

Resources

Log In

Sign Up

Task Category

conditional-text-generation

text-classification

structure-prediction

sequence-modeling

question-answering

text-scoring

+ 3

Task

machine-translation

language-modeling

named-entity-recognition

sentiment-classification

dialogue-modeling

extractive-qa

+ 128

Language

en

es

fr

de

ru

ar

+ 184

Multilinguality

monolingual

multilingual

translation

other-language-learner

Size

10K<n<100K

1K<n<10K

n<1K

100K<n<1M

n>1M

1k<10K

+ 18

License

mit

cc-by-4.0

cc-by-sa-4.0

cc-by-sa-3.0

apache-2.0

cc-by-nc-4.0

+ 56

Datasets 638

Search Datasets

Sort: Alphabetical

acronym_identification

Acronym identification training and development sets for the acronym identification task at SDU@AAAI-21.

annotations_creators: expert-generated

language_creators: found

languages: en

licenses: mit

multilinguality: monolingual

size_categories: 10K<n<100K

source_datasets: original

task_categories: structure-prediction

task_ids: structure-prediction-other-acronym-identification

ade_corpus_v2

ADE-Corpus-V2 Dataset: Adverse Drug Reaction Data. This is a dataset for Classification if a sentence is ADE-related (True) or not (False) and Relation Extraction between Adverse Drug Event and Drug. DRUG-AE.rel provides relations between drugs and adverse effects. DRUG-DOSE.rel provides relations between drugs and dosages. ADE-NEG.txt pro...

annotations_creators: expert-generated

language_creators: found

languages: en

licenses: unknown

multilinguality: monolingual

size_categories: 10K<n<100K

size_categories: 1K<n<10K

size_categories: n<1K

source_datasets: original

task_categories: text-classification

task_categories: structure-prediction

task_categories: structure-prediction

task_ids: fact-checking

task_ids: coreference-resolution

task_ids: coreference-resolution

adversarial_qa

AdversarialQA is a Reading Comprehension dataset, consisting of questions posed by crowdworkers on a set of Wikipedia articles using an adversarial model-in-the-loop. We use three different models; BiDAF (Seo et al., 2016), BERT-Large (Devlin et al., 2018), and RoBERTa-Large (Liu et al., 2019) in the annotation loop and construct three datasets;...

annotations_creators: crowdsourced

language_creators: found

languages: en

licenses: cc-by-sa-4.0

multilinguality: monolingual

size_categories: 10K<n<100K

source_datasets: original

task_categories: question-answering

task_ids: extractive-qa

task_ids: open-domain-qa



Paperswithcode Datasets

- <https://www.paperswithcode.com/datasets?mod=texts&page=1>

835 dataset results for Texts x



Penn Treebank

The English Penn Treebank corpus, and in particular the section of the corpus corresponding to the articles of Wall Street Journal (WSJ), is one of the most known and used corpus for t...
1,545 PAPERS • 10 BENCHMARKS



SQuAD (Stanford Question Answering Dataset)

The Stanford Question Answering Dataset (SQuAD) is a collection of question-answer pairs derived from Wikipedia articles. In SQuAD, the correct answers of questions can be any se-...
1,254 PAPERS • 7 BENCHMARKS



Visual Genome

Visual Genome contains Visual Question Answering data in a multi-choice setting. It consists of 101,174 images from MSCOCO with 1.7 million QA pairs, 17 questions per image on aver-...
903 PAPERS • 11 BENCHMARKS



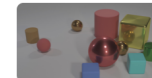
GLUE (General Language Understanding Evaluation benchmark)

General Language Understanding Evaluation (GLUE) benchmark is a collection of nine natural language understanding tasks, including single-sentence tasks CoLA and SST-2, similarity...
847 PAPERS • 14 BENCHMARKS



SNLI (Stanford Natural Language Inference)

The SNLI dataset (Stanford Natural Language Inference) consists of 570k sentence-pairs manually labeled as entailment, contradiction, and neutral. Premises are image captions fro-...
743 PAPERS • 1 BENCHMARK



CLEVR (Compositional Language and Elementary Visual Reasoning)

CLEVR (Compositional Language and Elementary Visual Reasoning) is a synthetic Visual Question Answering dataset. It contains images of 3D-rendered objects; each image comes...
528 PAPERS • 1 BENCHMARK



Visual Question Answering (VQA)

Visual Question Answering (VQA) is a dataset containing open-ended questions about images. These questions require an understanding of vision, language and commonsense...
435 PAPERS • 2 BENCHMARKS



Billion Word Benchmark

The One Billion Word dataset is a dataset for language modeling. The training/held-out data was produced from the WMT 2011 News Crawl data using a combination of Bash shell and...
417 PAPERS • 1 BENCHMARK

Machine translation

- <http://statmt.org>
- Look in particular at the various WMT shared tasks

Sitemap

- [SMT Book](#)
- [Research Survey Wiki](#)
- [Moses MT System](#)
- [Europarl Corpus](#)
- [News Commentary Corpus](#)
- [Online Evaluation](#)
- [Online Moses Demo](#)
- [Translation Tool](#)
- [WMT Workshop 2014](#)
- [WMT Workshop 2013](#)
- [WMT Workshop 2012](#)
- [WMT Workshop 2011](#)
- [WMT Workshop 2010](#)
- [WMT Workshop 2009](#)
- [WMT Workshop 2008](#)
- [WMT Workshop 2007](#)
- [WMT Workshop 2006](#)
- [WMT Workshop 2005](#)

Statistical Machine Translation

This website is dedicated to research in statistical machine translation, i.e. the translation of text from one human language to another by a computer that learned how to translate from vast amounts of translated text.

Introduction to Statistical MT Research

- [The Mathematics of Statistical Machine Translation](#) by Brown, Della Petra, Della Pietra, and Mercer
- [Statistical MT Handbook](#) by Kevin Knight
- [SMT Tutorial \(2003\)](#) by Kevin Knight and Philipp Koehn
- ESSLI Summer Course on SMT (2005), [day1](#), [2](#), [3](#), [4](#), [5](#) by Chris Callison-Burch and Philipp Koehn.
- [MT Archive](#) by John Hutchins, electronic repository and bibliography of articles, books and papers on topics in machine translation and computer-based translation tools

Dependency parsing: Universal Dependencies

- <https://universaldependencies.org>

Universal Dependencies

Universal Dependencies (UD) is a framework for cross-linguistically consistent grammatical annotation and an open community effort with over 200 contributors producing more than 100 treebanks in over 70 languages.

- [Short introduction to UD](#)
- [UD annotation guidelines](#)
- More information on UD:
 - [How to contribute to UD](#)
 - [Tools for working with UD](#)
 - [Discussion on UD](#)
 - [UD-related events](#)
- Query UD treebanks online:
 - [SETS treebank search](#) maintained by the University of Turku
 - [PML Tree Query](#) maintained by the Charles University in Prague
 - [Kontext](#) maintained by the Charles University in Prague
 - [Grew-match](#) maintained by Inria in Nancy
- [Download UD treebanks](#)

If you want to receive news about Universal Dependencies, you can subscribe to the [UD mailing list](#). If you want to discuss individual annotation questions, use the [Github issue tracker](#).

Many, many more

- There are now many other datasets available online for all sorts of purposes
 - Look at Kaggle
 - Look at research papers to see what data they use
 - Look at lists of datasets
 - <https://machinelearningmastery.com/datasets-natural-language-processing/>
 - <https://github.com/niderhoff/nlp-datasets>
 - Lots of particular things:
 - <https://gluebenchmark.com/tasks>
 - <https://nlp.stanford.edu/sentiment/>
 - <https://research.fb.com/downloads/babi/> (Facebook bAbI-related)
 - Ask on Ed or talk to course staff

5. Care with datasets and in model development

- Many publicly available datasets are released with a **train/dev/test** structure.
- **We're all on the honor system to do test-set runs only when development is complete. Do development testing on the dev set only!**
- Splits like this presuppose a fairly large dataset.
- If there is no dev set or you want a separate tune set, then you create one by splitting the training data
 - We have to weigh the usefulness of it being a certain size against the reduction in train-set size.
 - **Cross-validation** (q.v.) is a technique for maximizing data when you don't have much
- Having a fixed test set ensures that all systems are assessed against the same gold data. This is generally good, but it is problematic when the test set turns out to have unusual properties that distort progress on the task.

Training models and pots of data

- We are always interested in **generalization performance** on an **independent test set**, not seen or used during training
- You build (estimate/train) a model on a **training set**.
- Often, you then set further hyperparameters on another, independent set of data, the **tuning set**
 - The tuning set is the training set for the hyperparameters!
- You measure progress as you go on a **dev set** (development test set or validation set)
 - If you do that a lot you overfit to the dev set so it can be good to have a second dev set, the **dev2** set
- **Only at the end**, you evaluate and present final numbers on a **test set**
 - Use the final test set **extremely** few times ... ideally only once

Training models and pots of data

- The **train**, **tune**, **dev**, and **test** sets need to be completely distinct
- It is invalid to give results testing on material you have trained on
 - You **will** get a falsely good performance.
 - We almost always overfit on train
- You need an independent tuning set
 - The hyperparameters won't be set right if tune is same as train
- If you keep running on the same evaluation set, you begin to overfit to that evaluation set
 - Effectively you are “training” on the evaluation set ... you are learning things that do and don't work on that particular eval set and using the info
- To get a valid measure of system performance you need another untrained on, **independent** test set ... hence dev2 and final test

Getting your neural network to train

- Start with a positive attitude!
 - **Neural networks want to learn!**
 - If the network isn't learning, you're doing something to prevent it from learning successfully
- Realize the grim reality:
 - **There are lots of things that can cause neural nets to not learn at all or to not learn very well**
 - Finding and fixing them (“debugging and tuning”) can often take more time than implementing your model
- It's hard to work out what these things are
 - But experience, experimental care, and rules of thumb help!

Experimental strategy

- Work incrementally!
- Start with a very simple model and get it to work!
 - It's hard to fix a complex but broken model
- Add bells and whistles one-by-one and get the model working with each of them (or abandon them)
- Initially run on a tiny amount of data
 - You will see bugs much more easily on a tiny dataset ... and they train really quickly
 - Something like 4–8 examples is good
 - Often synthetic data is useful for this
 - Make sure you can get 100% on this data (testing on train)
 - Otherwise your model is definitely either not powerful enough or it is broken

Experimental strategy

- Train and run your model on a large dataset
 - It should still score close to 100% on the training data after optimization
 - Otherwise, you probably want to consider a more powerful model!
 - Overfitting to training data is **not** something to fear when doing deep learning
 - These models are usually good at generalizing because of the way distributed representations share statistical strength regardless of overfitting to training data
- But, still, you now want good generalization performance:
 - Regularize your model until it doesn't overfit on dev data
 - Strategies like L2 regularization can be useful
 - But normally **generous dropout** is the secret to success

Details matter!

- Look at your data, collect summary statistics
- Look at your model's outputs, do error analysis
- Tuning hyperparameters, learning rates, getting initialization right, etc. is **often** important to the successes of neural nets

Good luck with your projects!